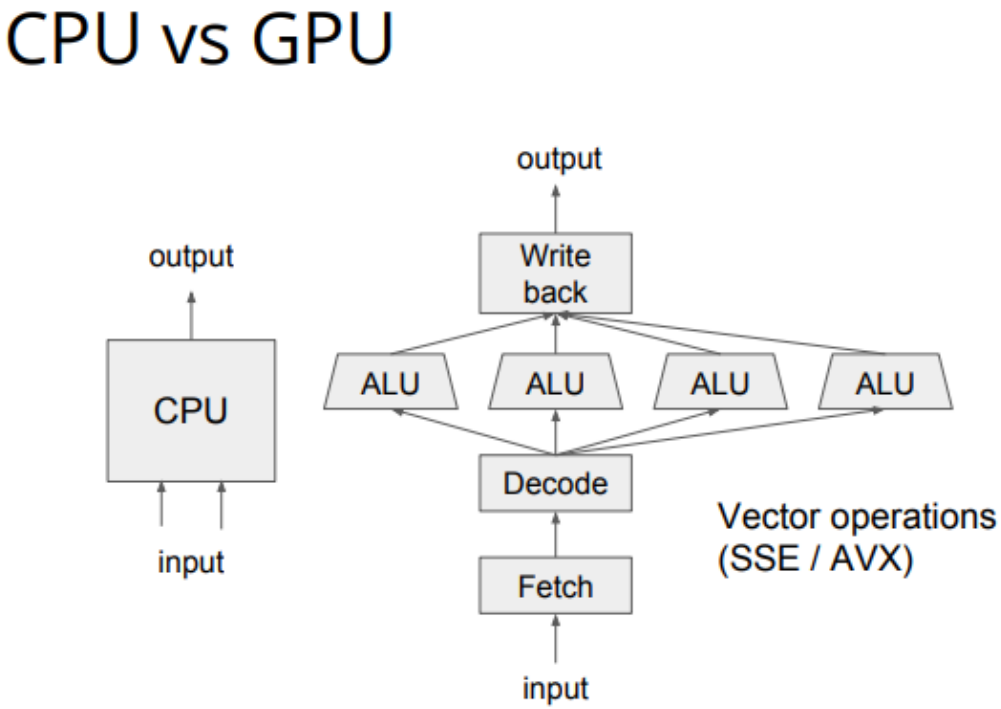
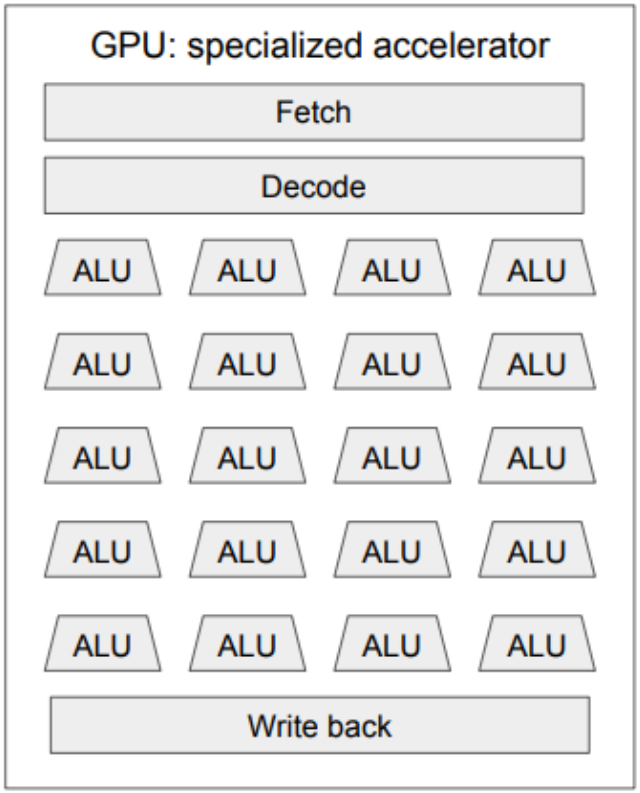
Review

Graphics Processing Unit

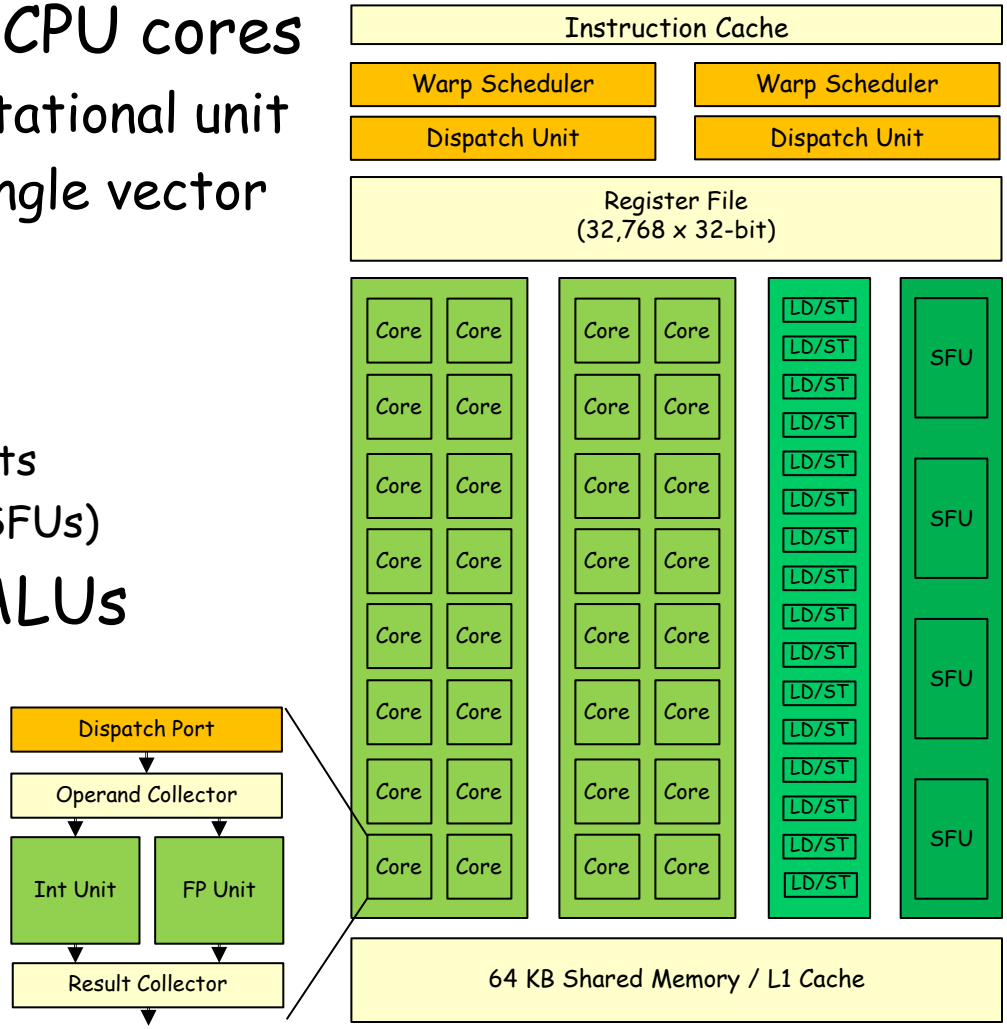
CPU vs. GPU


CPU vs GPU
output
CPU
input
output
Write back
ALU
ALU
ALU
ALU
Decode
Fetch
input
Vector operations
(SSE / AVX)


GPU: specialized accelerator
Fetch
Decode
ALU
Write back

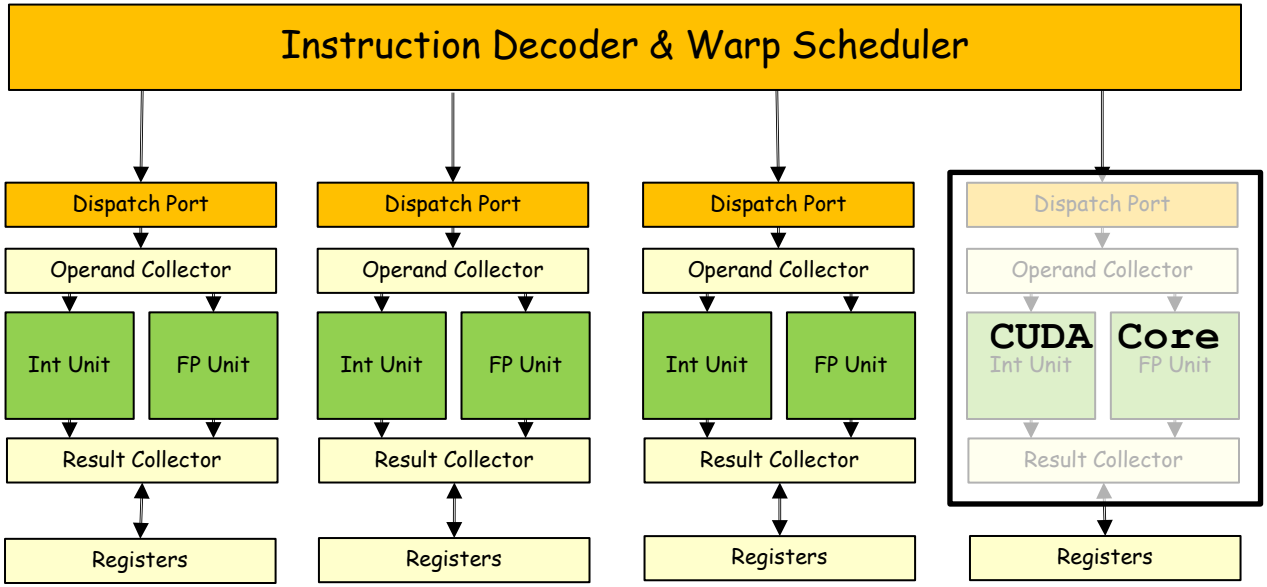
Streaming Multiprocessor

- SMs are analogous to CPU cores
  - SM is the basic computational unit
  - Consider an SM as a single vector processor
  - Composed of multiple
    - CUDA "Cores"
    - Load/Store (LD/ST) units
    - Special function units (SFUs)
- CUDA Core contains ALUs
  - Integer ALU
  - Floating-point ALU



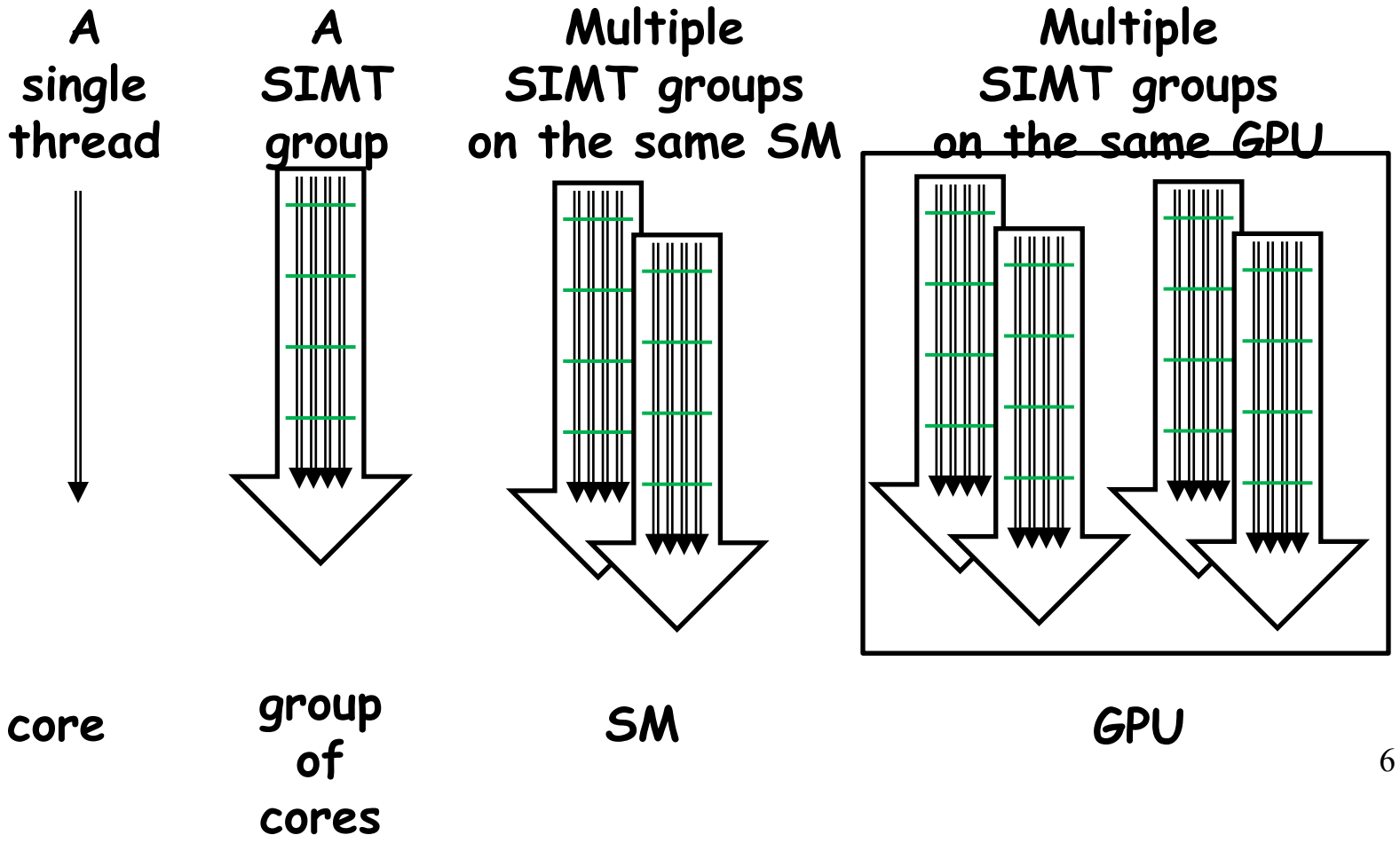
SIMT Execution Model

- Single Instruction Multiple Threads
  - The instruction only decodes once
  - All "cores" execute the same instructions
    - But operate on their own private registers
  - A SIMT group names a "warp" (32 threads)



SIMT Execution Model

- Together, GPU has a nested thread hierarchy



CUDA Threading Model

- CUDA programs consist of a hierarchy of concurrent threads
  - Grid – Block – Thread
    - A grid consists of multiple blocks
    - A block consists of multiple threads
  - Launch a kernel means launches a grid of blocks

```
vectorAdd<<<16, 64>>>(d_A, d_B, d_C, length);
```

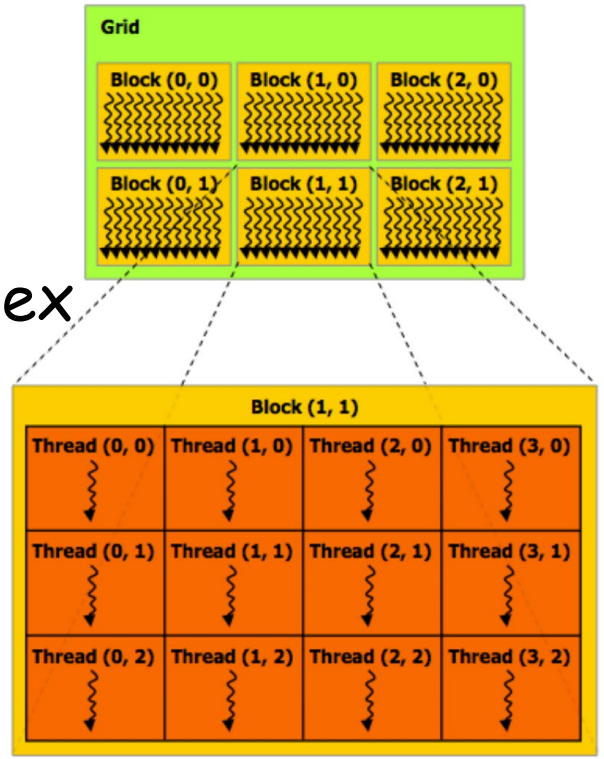
16 blocks

64 threads per block

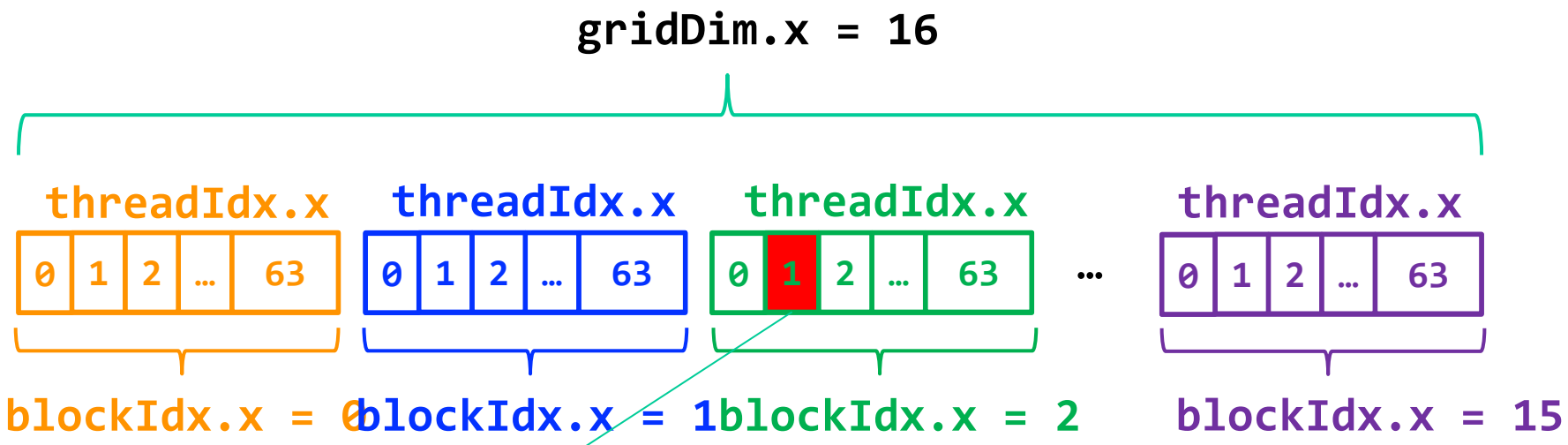
CUDA Threading Model

- Grid/Block also can be up to 3-dimensional
  - 1-dimensional at default
  - gridDim.x, gridDim.y, gridDim.z
  - blockDim.x, blockDim.y, blodkDim.z

- Each Block/Thread has it's unique index
  - Each block can be identified by
    - blockIdx.x, blockIdx.y, blockIdx.z
  - Each thread can be identified by
    - blockIdx.x, blockIdx.y, blockIdx.z
    - threadIdx.x, threadIdx.y, threadIdx.z



Indexing Arrays with Blocks and Threads

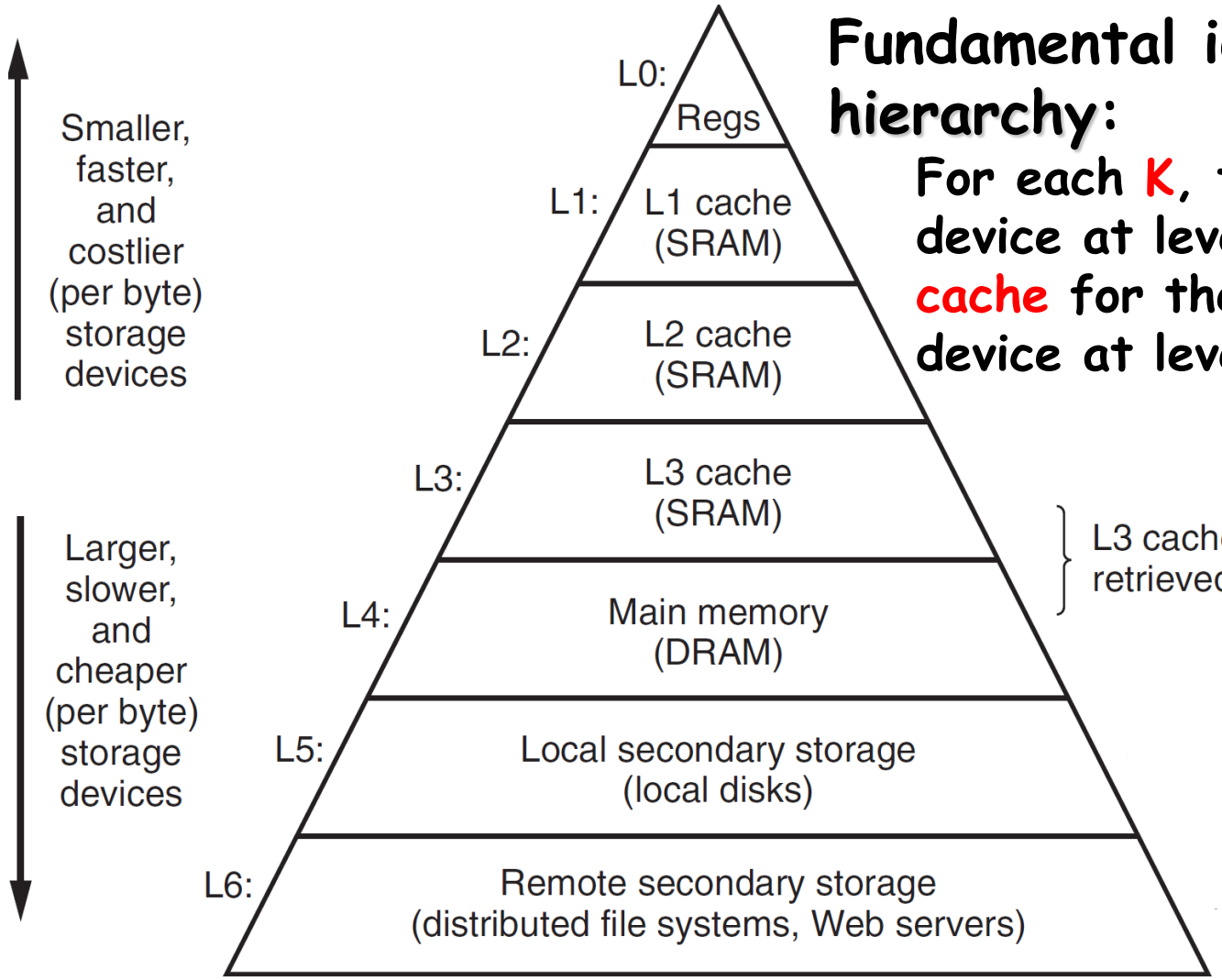



```
total_threads = gridDim.x * blockDim.x
total_threads =  (16)   *  (64)
```

```
thread_id = blockDim.x * blockIdx.x + threadIdx.x
thread_id =  (64)   *   (2)  +   (1)
```

Memory Hierarchy

Caches



Fundamental idea of a memory hierarchy:

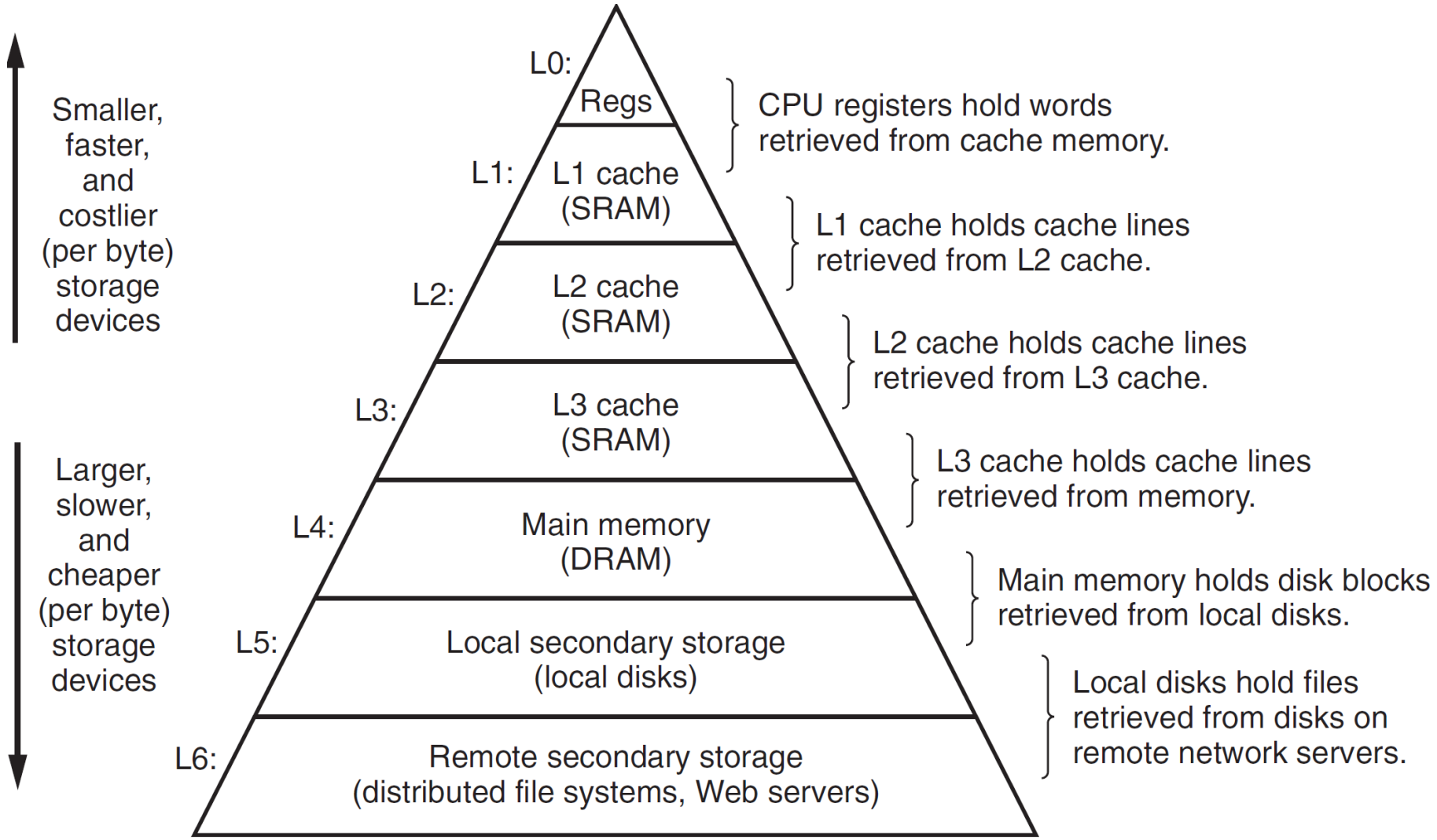
For each K, the faster, smaller device at level K serves as a cache for the larger, slower device at level K+1.

Caches

- Why do memory hierarchies work?
  - Because of locality, programs tend to access the data at level k more often than they access the data at level k+1.
  - Thus, the storage at level k+1 can be slower, and thus larger and cheaper per bit.

Big Idea: The memory hierarchy creates a large pool of storage that costs as much as the cheap storage near the bottom, but that serves data to programs at the rate of the fast storage near the top.

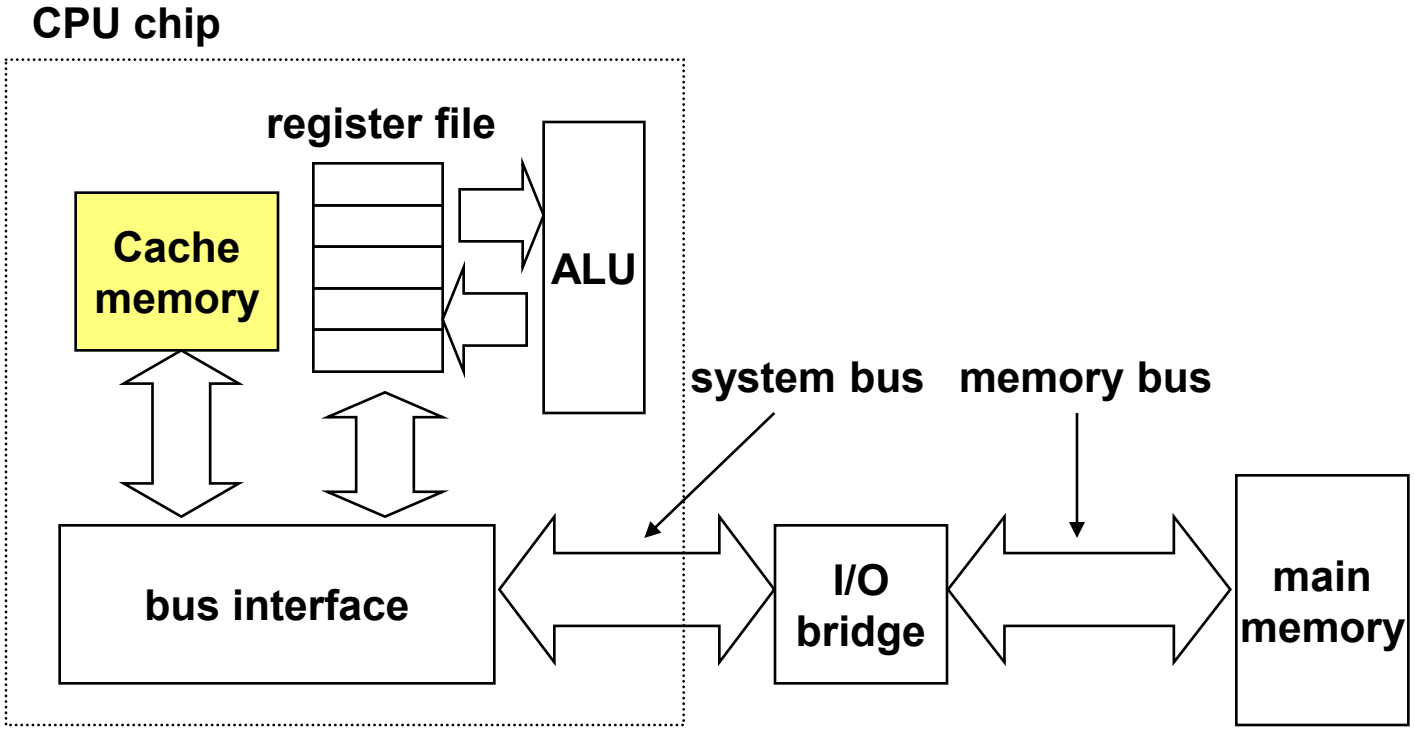
An example memory hierarchy



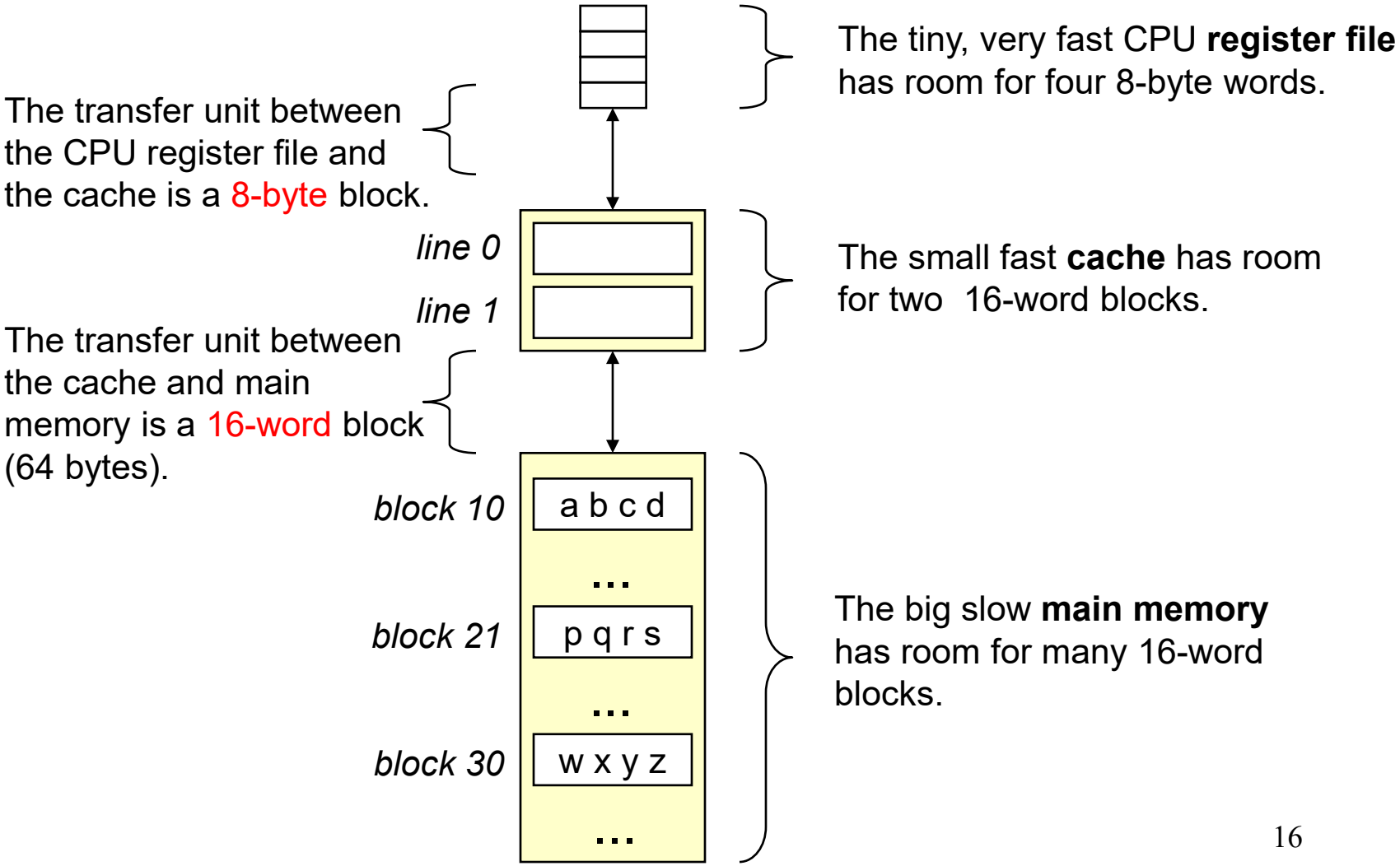
Cache Memory

Cache Memory

- Hold frequently accessed blocks of main memory in caches
- CPU looks first for data in L1, then in L2, then in L3, then in main memory



Inserting a cache between the CPU and main memory

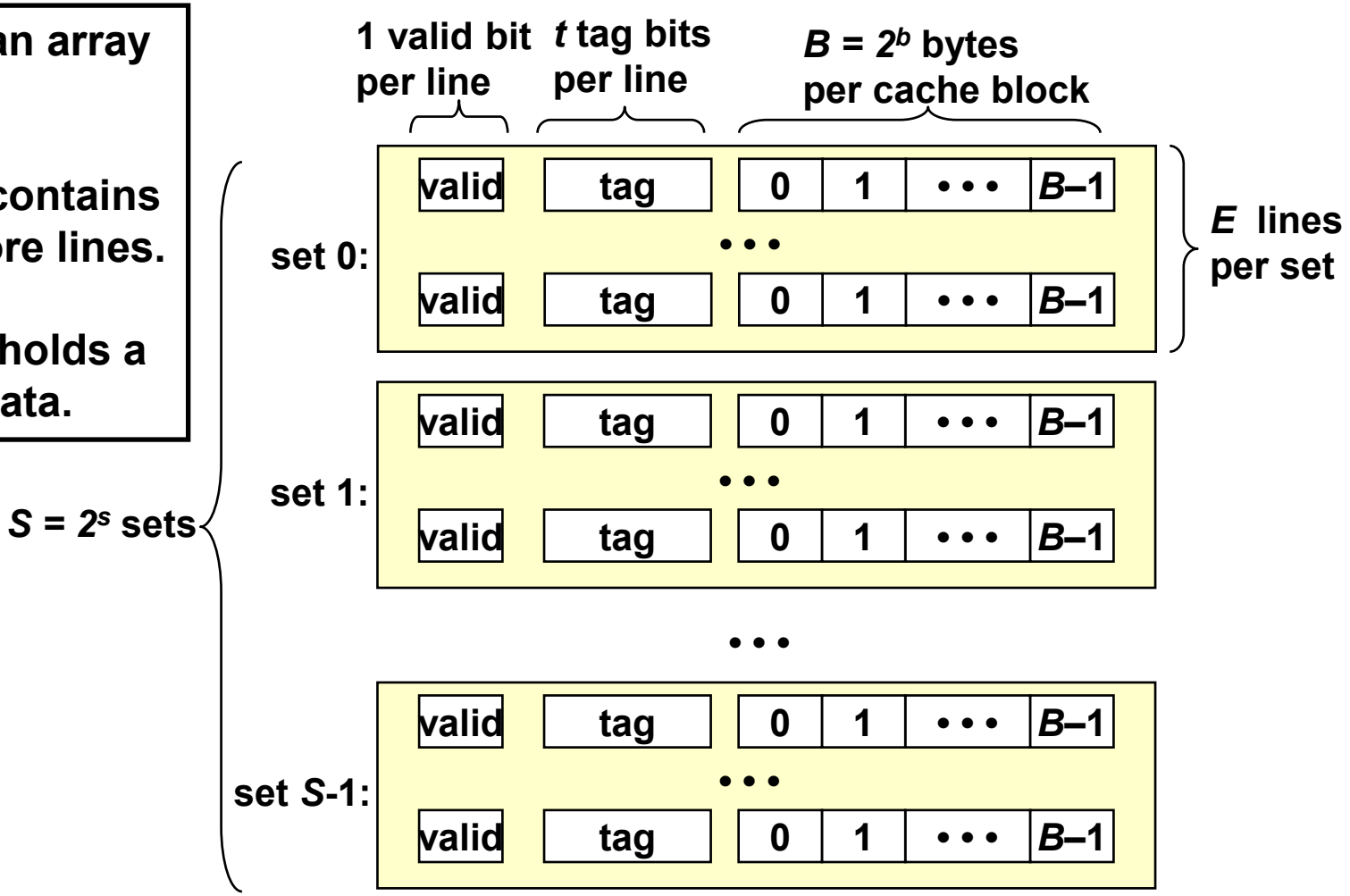


Generic Cache Memory Organization

Cache is an array of sets.

Each set contains one or more lines.

Each line holds a block of data.



Cache Memory

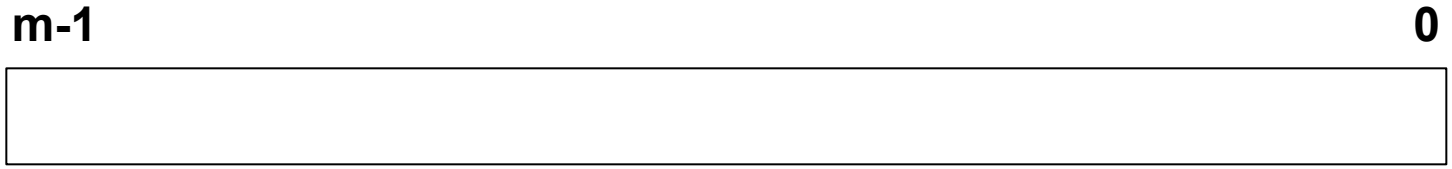
Fundamental parameters	
Parameters	Descriptions
$S = 2^s$	Number of sets
E	Number of lines per set
$B=2^b$	Block size(bytes)
$m=\log_2(M)$	Number of physical(main memory) address bits

Cache Memory

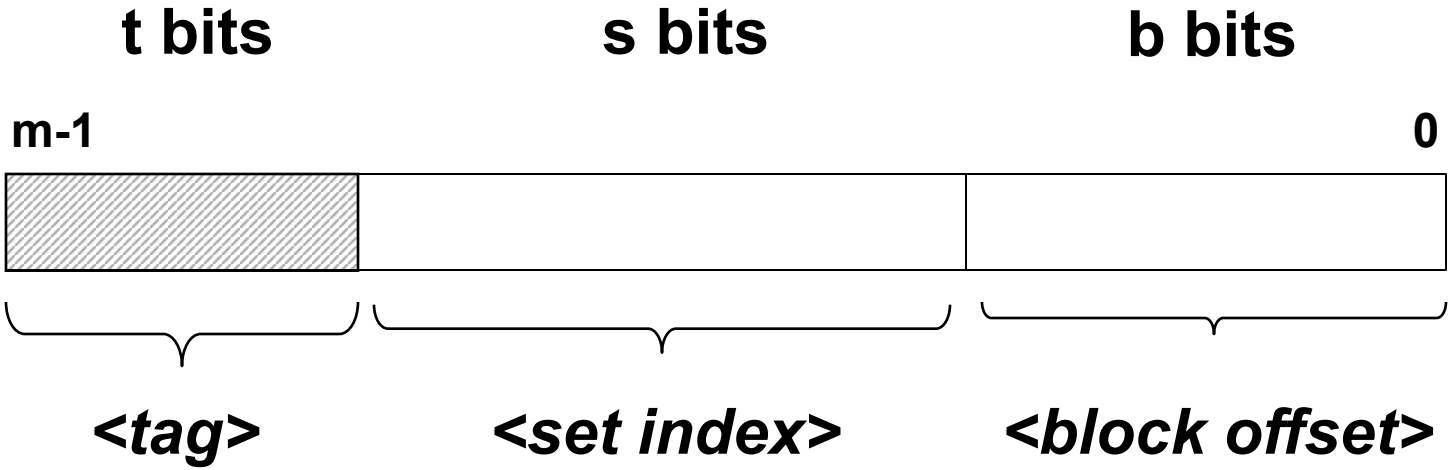
Derived quantities	
Parameters	Descriptions
$M=2^m$	Maximum number of unique memory address
s=log2(S)	Number of set index bits
b=log2(B)	Number of block offset bits
t=m-(s+b)	Number of tag bits
$C=B\times E\times S$	Cache size (bytes) not including overhead such as the valid and tag bits

Addressing caches

Physical Address A:

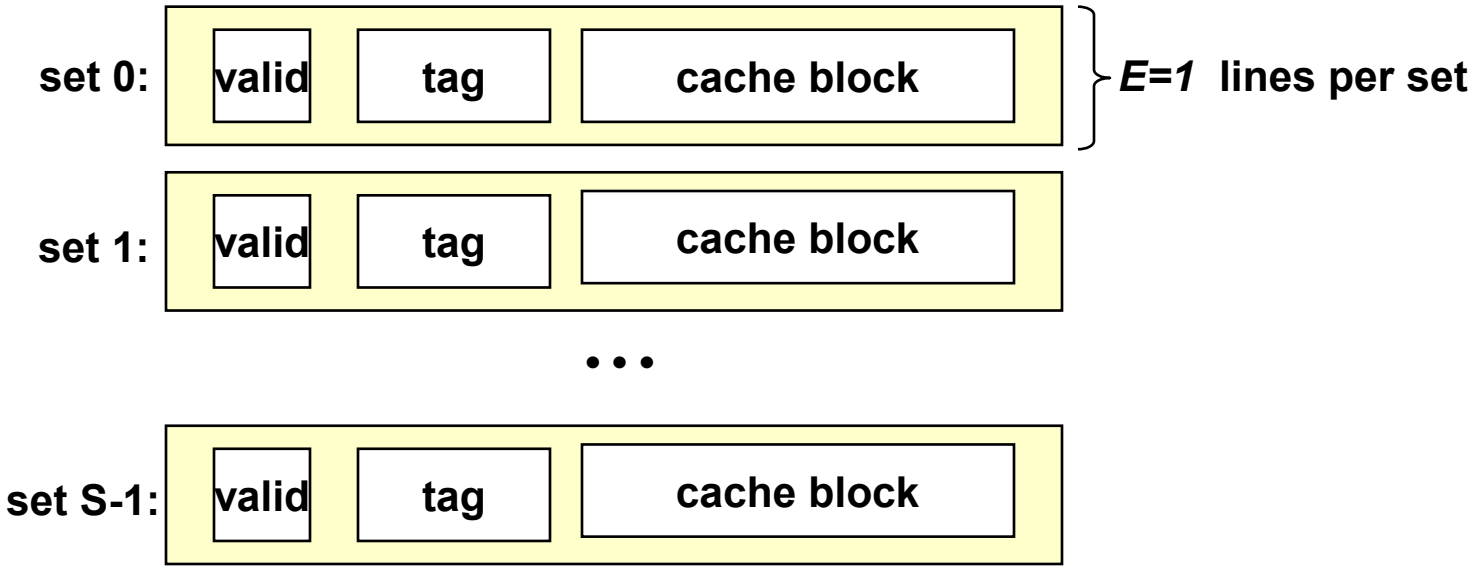


Split into 3 parts:



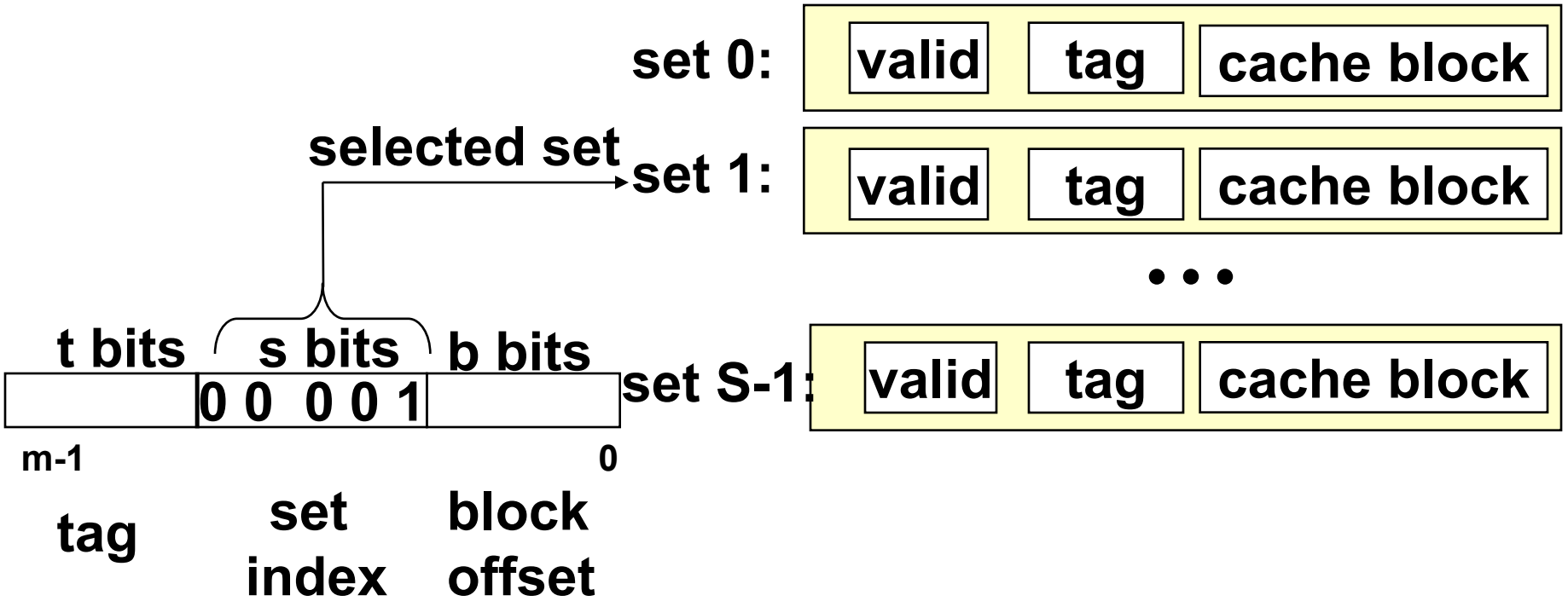
Direct-mapped cache

- Simplest kind of cache
- Characterized by exactly one line per set.



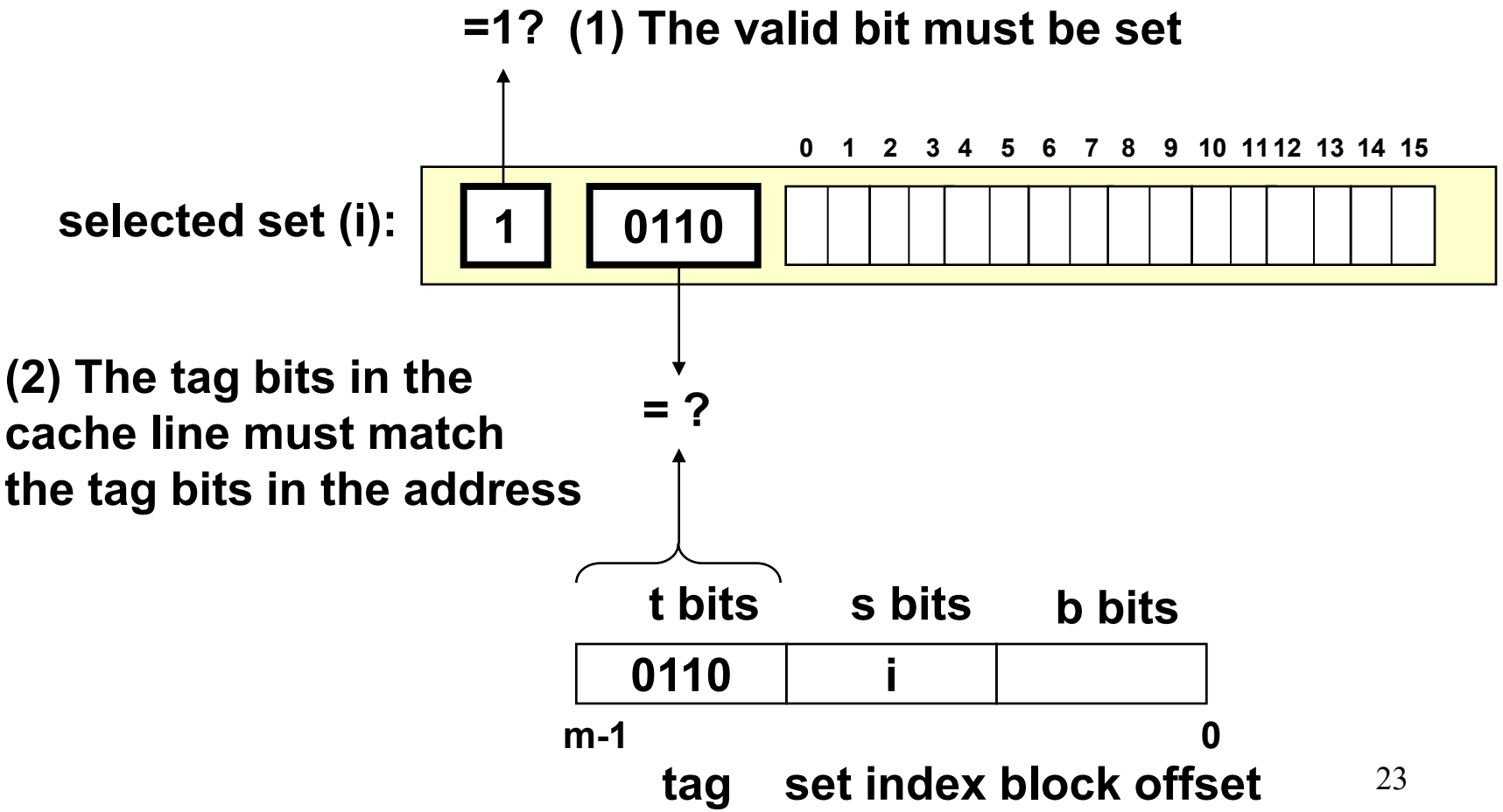
Set selection

- Use the set index bits to determine the set of interest

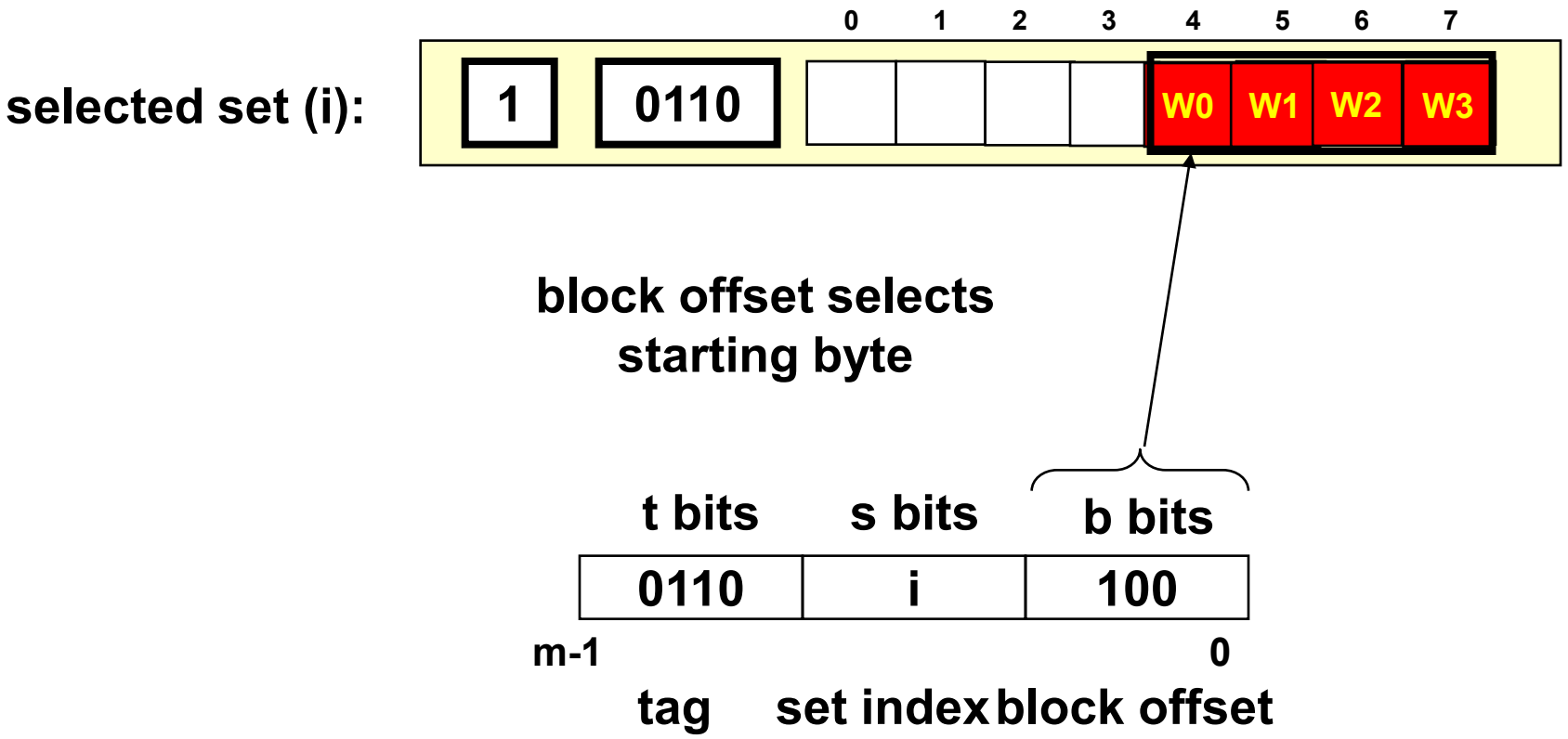


Line matching

Find a valid line in the selected set with a matching tag



Word Extraction



QUIZ

Problem1: Cache (31 points)

Consider a **16-bit machine** with a **2-way** set associative cache. The size of each block is **8 bytes**, and the total number of sets is 2. **LRU** policy is used for eviction and **write back** policy is used. The following table shows the content of the data cache at time T.

Set	Tag	Valid	Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0	0x0C0	1	0x2F	0x54	0x30	0xCC	0x66	0x9D	0xBC	0x7B
0	0xF0E	0	0xEE	0xD6	0xAC	0xAC	0xD0	0x0A	0xAC	0x9F
1	0x140	0	0x4C	0xAB	0xCD	0xEF	0xAD	0xBE	0xFF	0xEF
1	0x1E0	1	0x4B	0x19	0x24	0x25	0x99	0xDD	0x24	0xAF

1. What is the total size of the cache? (3')

   2*2*8=32bytes

2. How would a 16-bit physical memory address be split into tag/set-index/block-offset fields in this machine? (6')

   tag _12_ bits, set-index _1_ bits, and block-offset _3_ bits

3. A short program will read memory in the following sequences starting from time T. Each access will read **one byte**. Please fill the following blanks. If there is a cache miss, fill '-' for 'Byte Returned'. (10')

Order	Address	Hit/Miss	Byte Returned
1	0xF0E2	M	-
2	0x0C08	M	-
3	0x0C03	H	0xCC
4	0x2C3B	M	-
5	0x1E0A	M	0x24

One of the TA writes a program and tests it on this cache. The size of short type is **2 bytes**. The cache is **empty** before the execution of the program. Please only consider **data cache** access and ignore instruction cache access. Other processes will not interfere the status of memory and cache during this execution.

```
1. #define N 8
2. short A[N];
3. short B[N];
4. short C[N];
5.
6. void dot_product() {
7.     int i;
8.     for (i = 0; i < N; i++) {
9.         C[i] =  A[i] * B[i] + C[i];
10.    }
11.}
```

Assume that the address of A[0] is 0xa00. The address of B[0] is 0xb00. The address of C[0] is 0xc00. Answer the following questions:

4. Please calculate **the cache miss rate**. (3')

   3/4=75%

5. We can modify the cache through one of the following ways:

a) Double the number of sets while keeping the cache size unchanged.

b) Double the block size while keeping the cache size unchanged.

c) Increase the cache size via doubling the number of cache lines in each set.

Please choose the option(s) above that can optimize the miss rate of the original program and calculate the miss rate respectively.(6')

C 6/32=18.75%

6. Besides modifying the cache hardware parameters, can you propose a software-based solution to improve the cache hit rate? (3')

(Hint: Consider adjusting the starting addresses of arrays.)

Move the arrays in memory to ensure at least one of A[0], B[0], or C[0] lands in set0, and at least one in set1

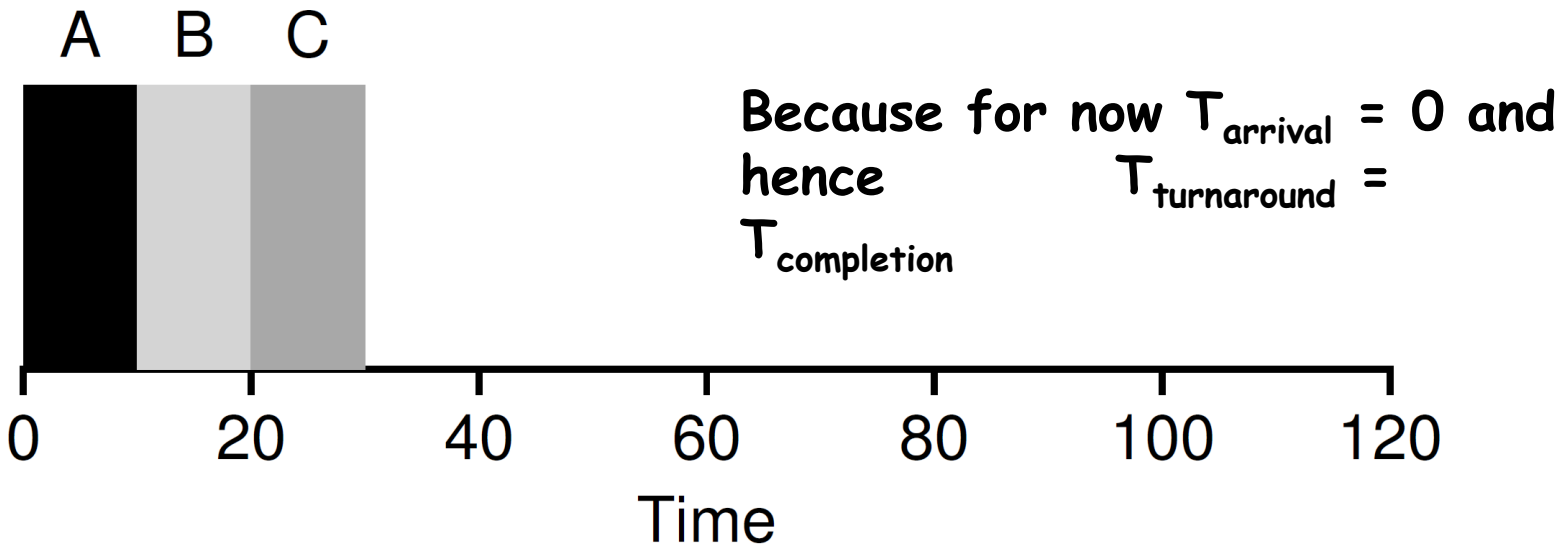
e.g. Move C to 0xC08

(The answer is not unique.)

Process Scheduling Introduction

First In, First Out (FIFO)

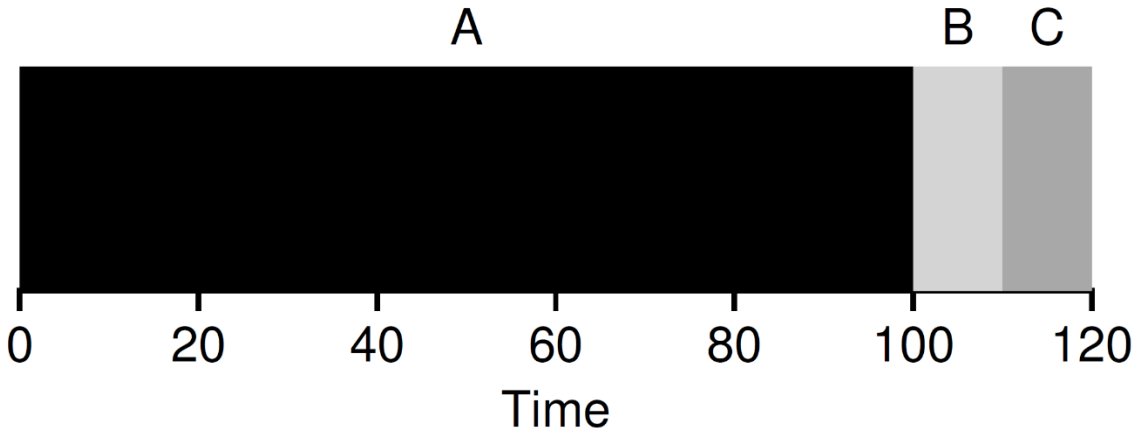
- Also known as First Come, First Served (FCFS)
- It is clearly simple and thus easy to implement
- Examples
  - Jobs A, B, C. Each runs for 10s
  - Turnaround time is (10+20+30)/3=20



Because for now $T_{arrival}$ = 0 and hence $T_{turnaround}$ = $T_{completion}$

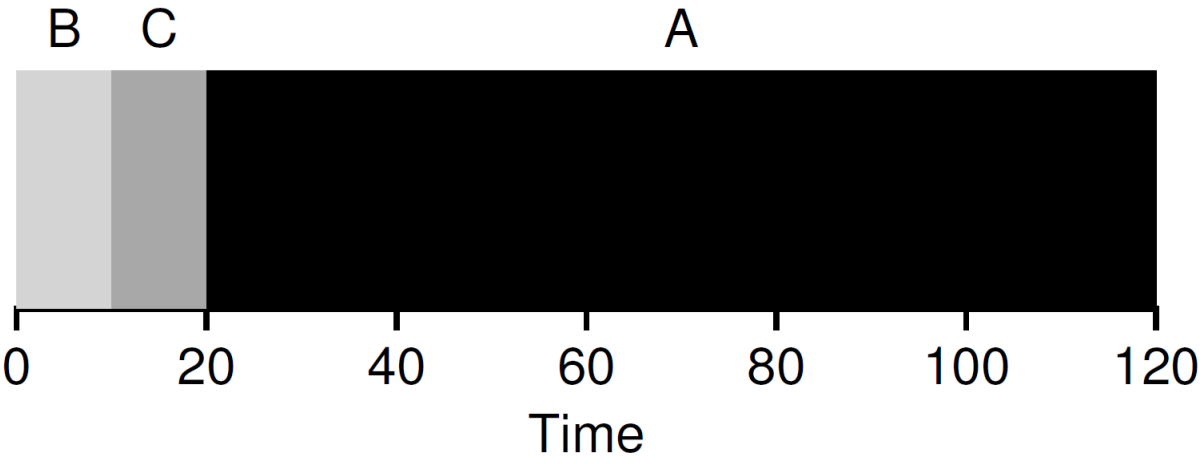
First In, First Out (FIFO)

- Job A runs for 100s. Job B and C run for 10s
  - Turnaround time is (100+110+120)/3=110
- Convey Effect
  - a number of relatively-short potential consumers of a resource get queued behind a heavyweight resource consumer



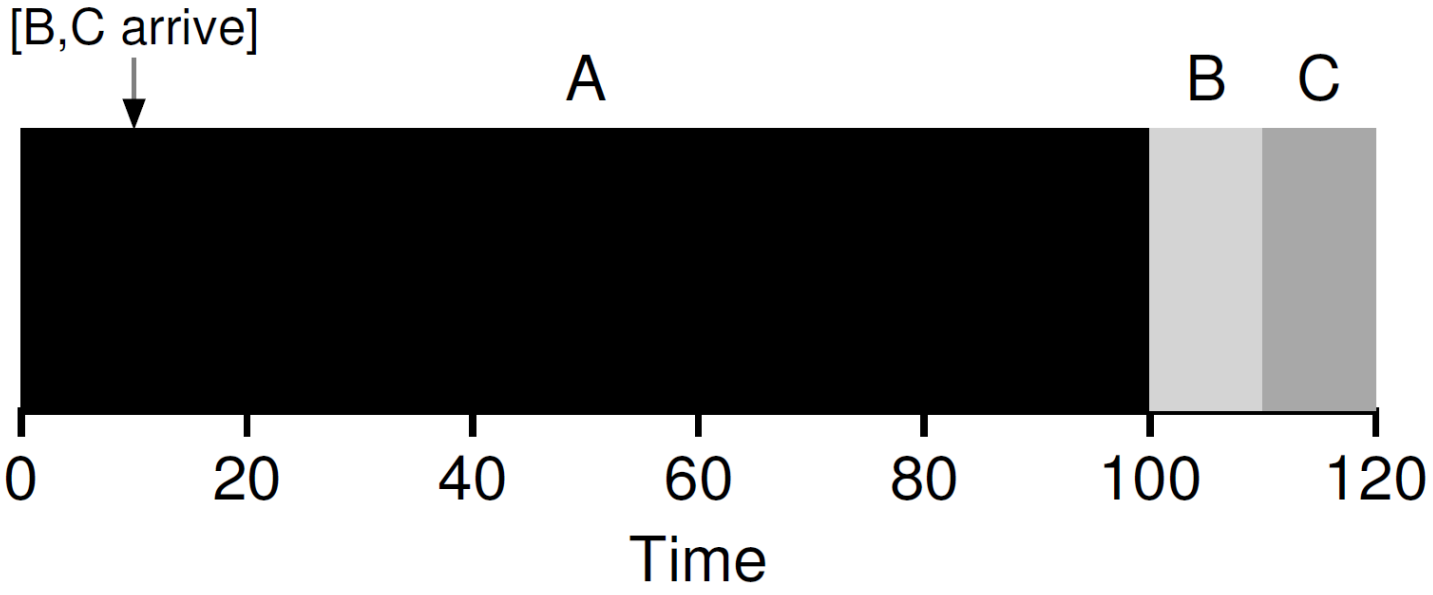
Shortest Job First (SJF)

- Runs the shortest job first
  - then the next shortest, and so on
  - optimal (throughput, turnaround time)
- Examples
  - Job A runs for 100s. Job B and C run for 10s
  - Turnaround time is (10+20+120)/3=50



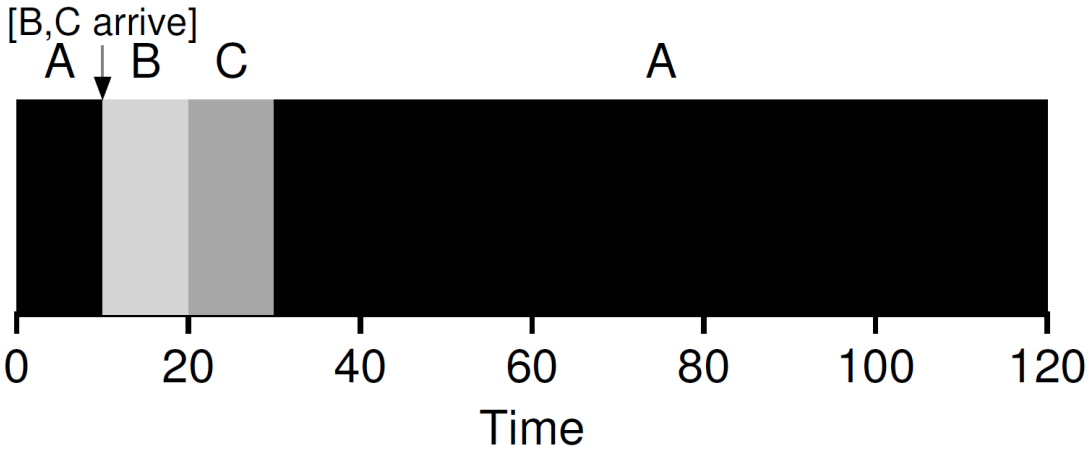
Shortest Job First (SJF)

- Assume A arrives at t = 0 and needs to run for 100 seconds, whereas B and C arrive at t = 10 and each need to run for 10 seconds
- Turnaround time (100+100+110)/3=103.33s



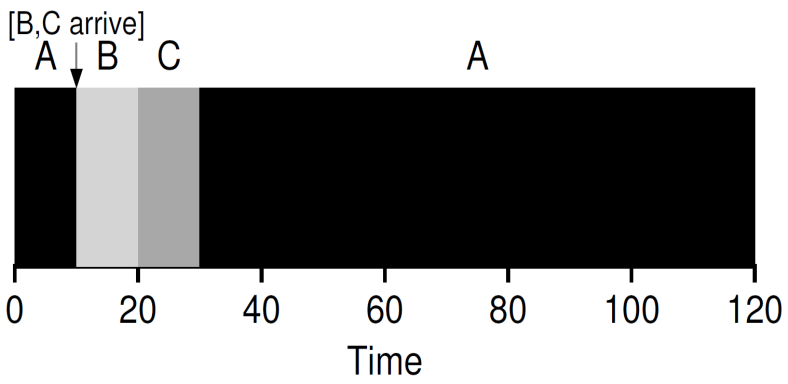
Shortest Time-to-Completion First (STCF)

- STCF
  - Any time a new job enters the system
  - The STCF scheduler determines
    - which of the remaining jobs (including the new job) has the least time left, and schedules that one
- Preempt A when B and C arrived
  - Turnaround time is (10+20+120)/3=50



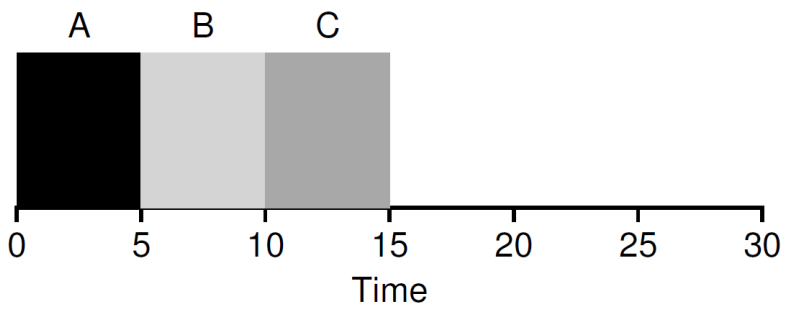
Another Metric: Response time

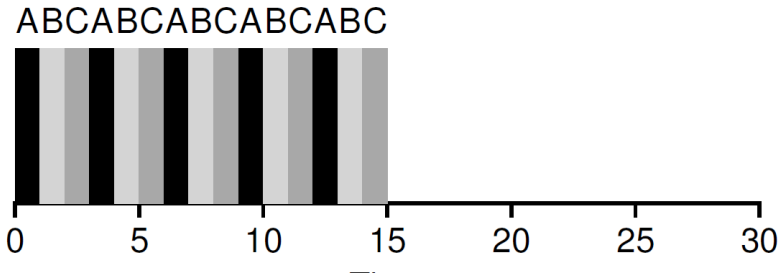
- In the time sharing systems
  - Many users are interactive with computers
  - Each user does not want to wait too long
- Response time
  - the time from when the job arrives in a system to the first time it is scheduled
  - $T_{response} = T_{firstturn} - T_{arrival}$
- Example
  - $T_{response} = (0+0+10)/3=3.33$



Round Rabin

- Mechanism
  - Run a job for a time slice
  - Switch to the next job in the run queue
  - Repeat until the jobs are finished
- Time slice
  - scheduling quantum
  - a multiple of the timer-interrupt period (10ms)





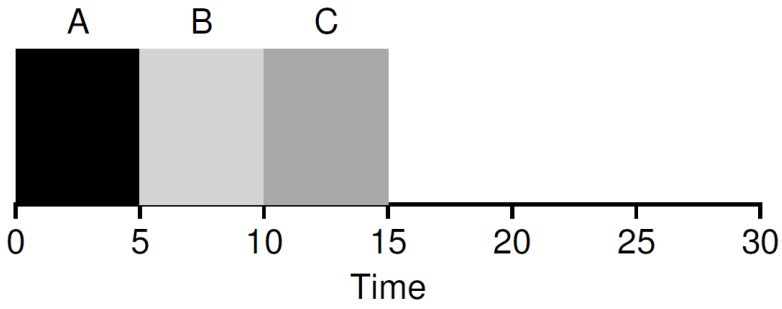
Round Rabin

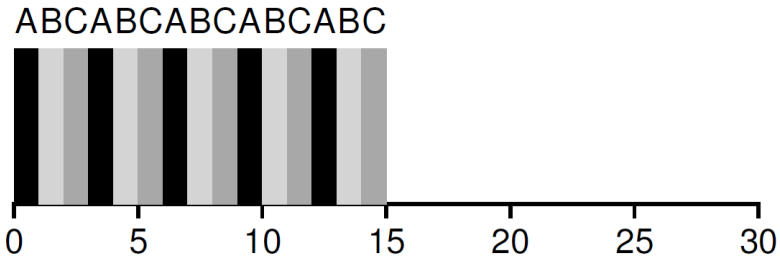
- Switching cost
  - Privilege switch, save and restore context
  - Flushing TLB and the hardware pipelines (or out of order execution states)

- Amortization
  - Do not switch too shortly
  - May increase the response time
  - Must do some trade-off

Round Rabin vs. SJF

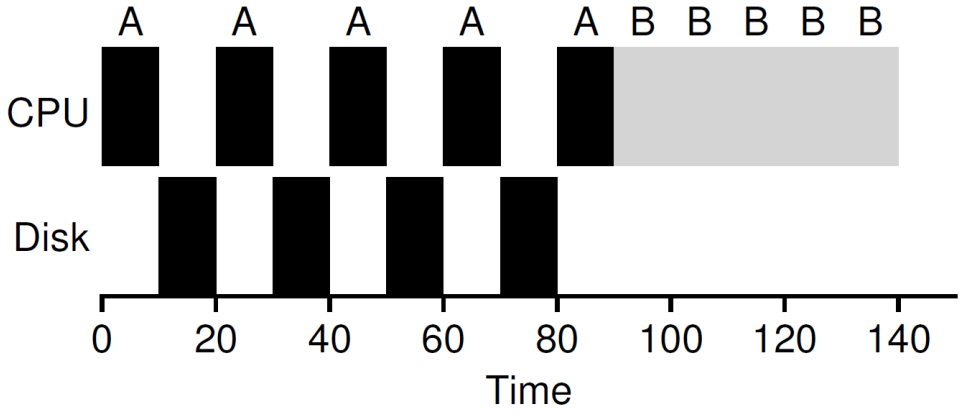
- SJF
  - Best for turnaround time
    - (5+10+15)/3=10
  - Worse for response time
    - (0+5+10)/3=5
- Round Rabin
  - Worse for turnaround time
    - (13+14+15)/3=14
  - Better for response time
    - (0+1+2)/3=1
- Must some trade-off





Incorporating I/O

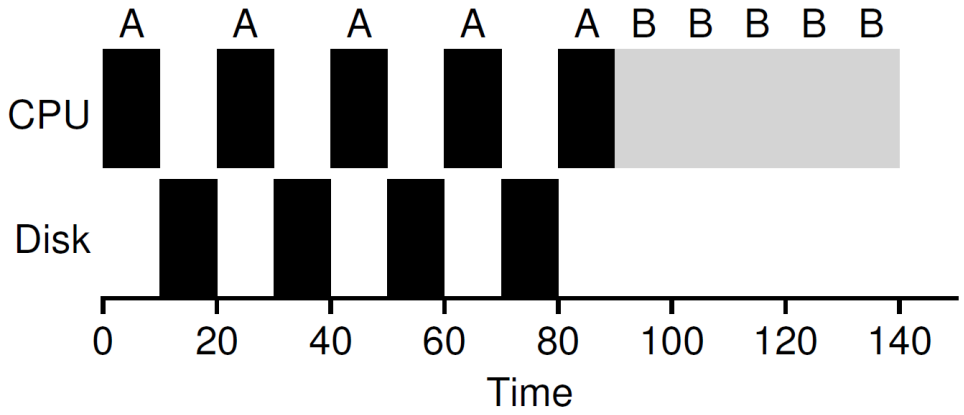
- Example
  - Job A and B run on CPU for 50s each
  - Job A does 4 I/O operations (10s each)
  - The poor schedule does as follows



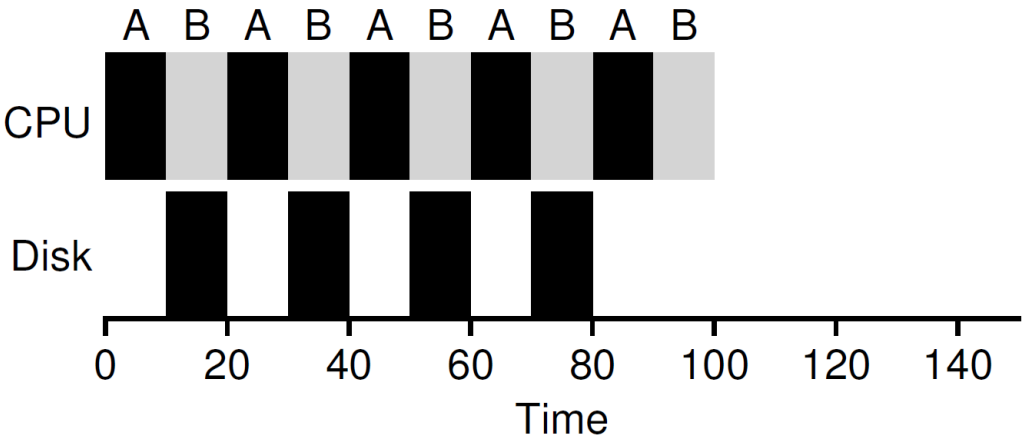
Incorporating I/O

- Example: Doing overlap better

No overlap



Overlap



MLFQ

- **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't).
- **Rule 2:** If Priority(A) = Priority(B), A & B run in RR.
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
- **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue.

EXE7

Problem 3: Scheduling Policy Comparison

Consider the workload below. Time 0 means the job runs during the interval [0 ms, 1 ms).

Job	Arrival (ms)	Run-time (ms)
A	0	8
B	2	2
C	4	1
D	6	4

Assume:

- **FIFO** is non-preemptive.
- **SJF** is non-preemptive (decisions made only at completion time).
- **STCF** is preemptive (re-evaluate on every arrival).
- **RR** uses a 2 ms quantum. When a quantum ends, any newly-arrived job is appended to the tail of the ready queue **before** the just-preempted job is appended.
- No I/O or context-switch cost.

3.1 Metrics table

Fill in the table below. Use exact fractions or decimals (e.g., `0.75`).

Policy	Avg. turnaround	Avg. response	CPU at t = 6 ms	CPU at t = 12 ms
FIFO	[a1]	[a2]	[a3]	[a4]
SJF	[b1]	[b2]	[b3]	[b4]
STCF	[c1]	[c2]	[c3]	[c4]
RR	[d1]	[d2]	[d3]	[d4]

3.2 Best response time

Which policy achieves the smallest average response time on this workload? Explain in 2–3 sentences why it does so well **and** identify one workload property that would make it perform worse than RR.

3.3 Convoy effect

Briefly describe the convoy effect using FIFO on this specific workload, citing the response-time numbers from the table.

Problem 4: MLFQ Trace

Consider the workload below, scheduled by **MLFQ** with two priority queues:

- **High priority** queue: 1 ms time-slice.
- **Low priority** queue: 2 ms time-slice.
- RR scheduling within each queue.
- A higher-priority arrival immediately preempts a running lower-priority job. The preempted job re-enters the tail of its queue and receives a **fresh** time-slice when it next runs.
- A job that uses up its full time-slice is demoted to the lower queue (jobs already on the low queue stay on the low queue and rotate via RR).
- **Priority boost is disabled.**
- New arrivals enter the high-priority queue.

Job	Arrival (ms)	Run-time (ms)
A	0	4
B	2	2
C	4	3

4.1 Fill the schedule

Complete the table below by writing the job that runs in each 1 ms interval.

Time (ms)	0	1	2	3	4	5	6	7	8
CPU	A	[a]	[b]	[c]	[d]	[e]	[f]	[g]	[h]

4.2 Compute metrics

Using your filled-in schedule, compute:

a. The completion time, turnaround time, and response time for each of A, B, C.
b. The average turnaround time and average response time across the three jobs.

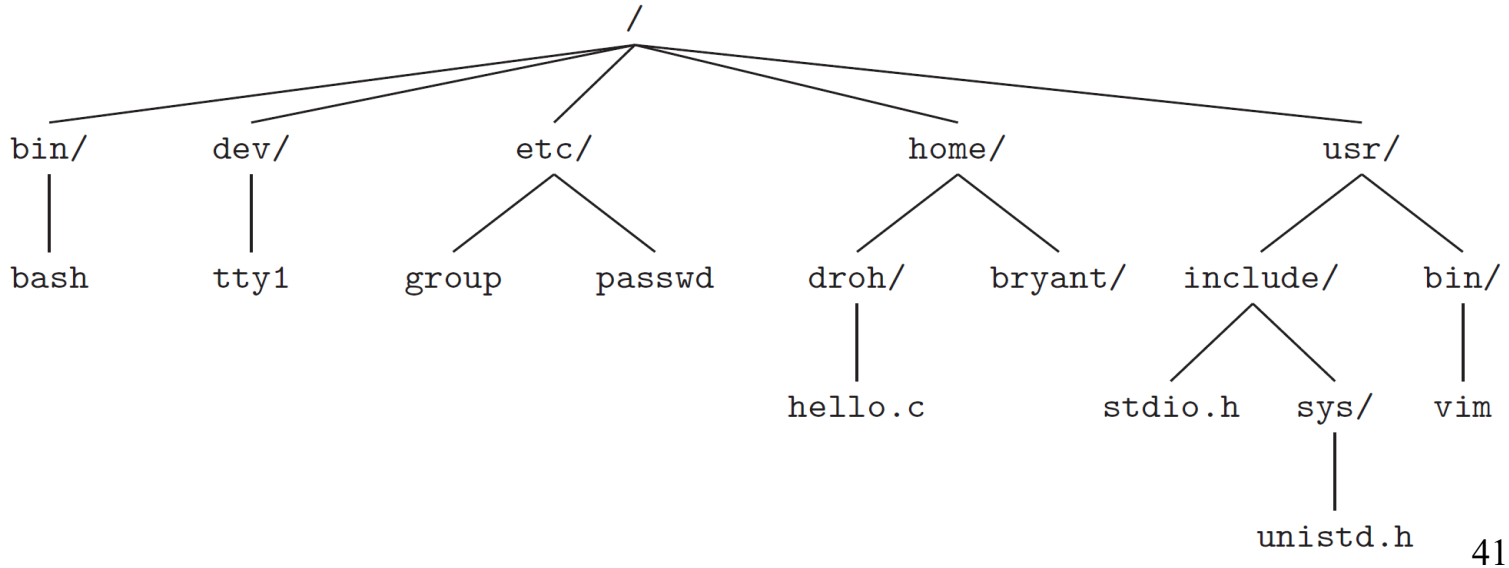
4.3 Starvation without priority boost

In 2–3 sentences, describe a workload (job arrivals + run-times) in which disabling priority boost causes one job to starve indefinitely under this 2-queue MLFQ. Be specific about which job starves and why.

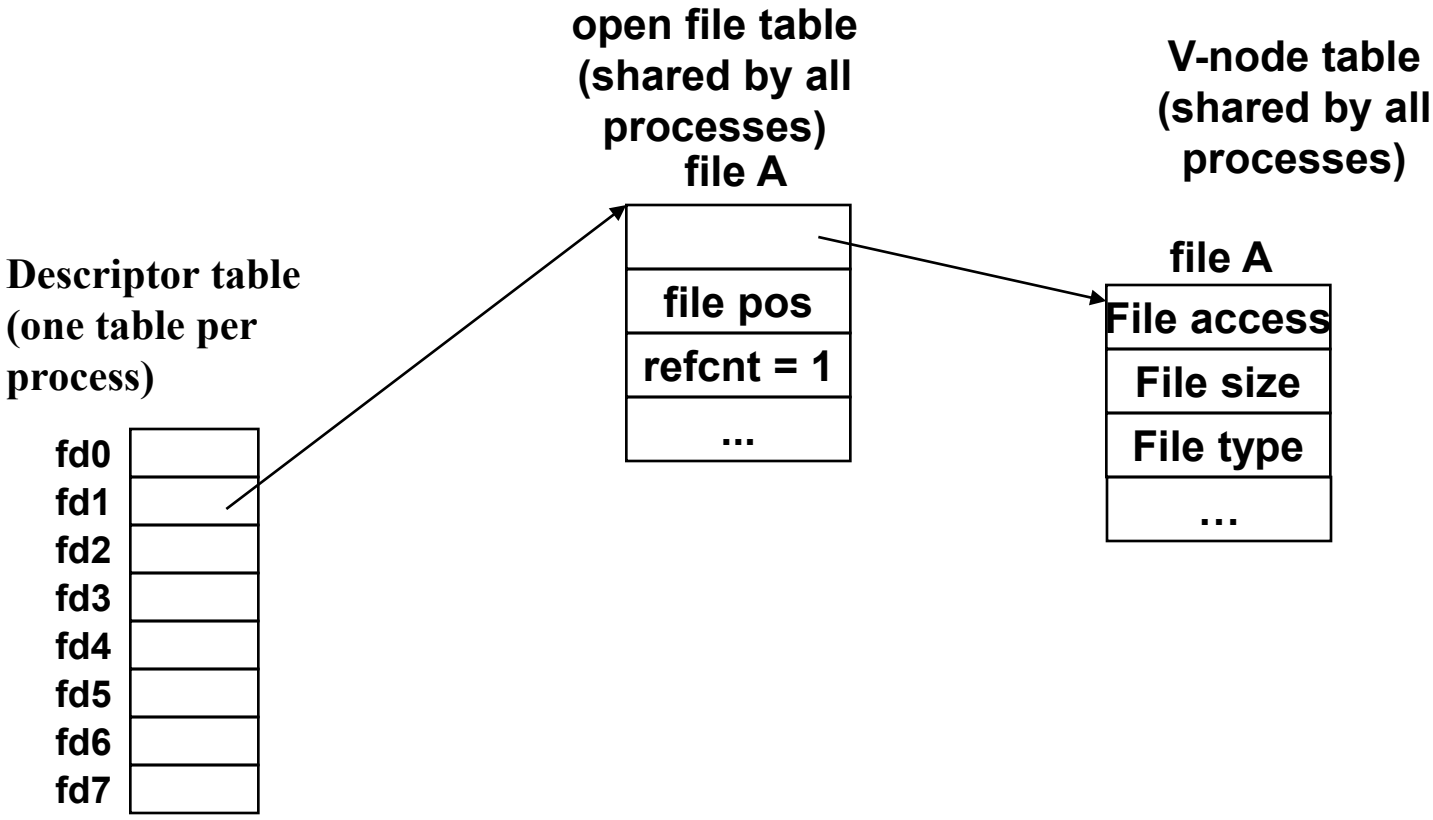
System-Level I/O

Directory Hierarchy

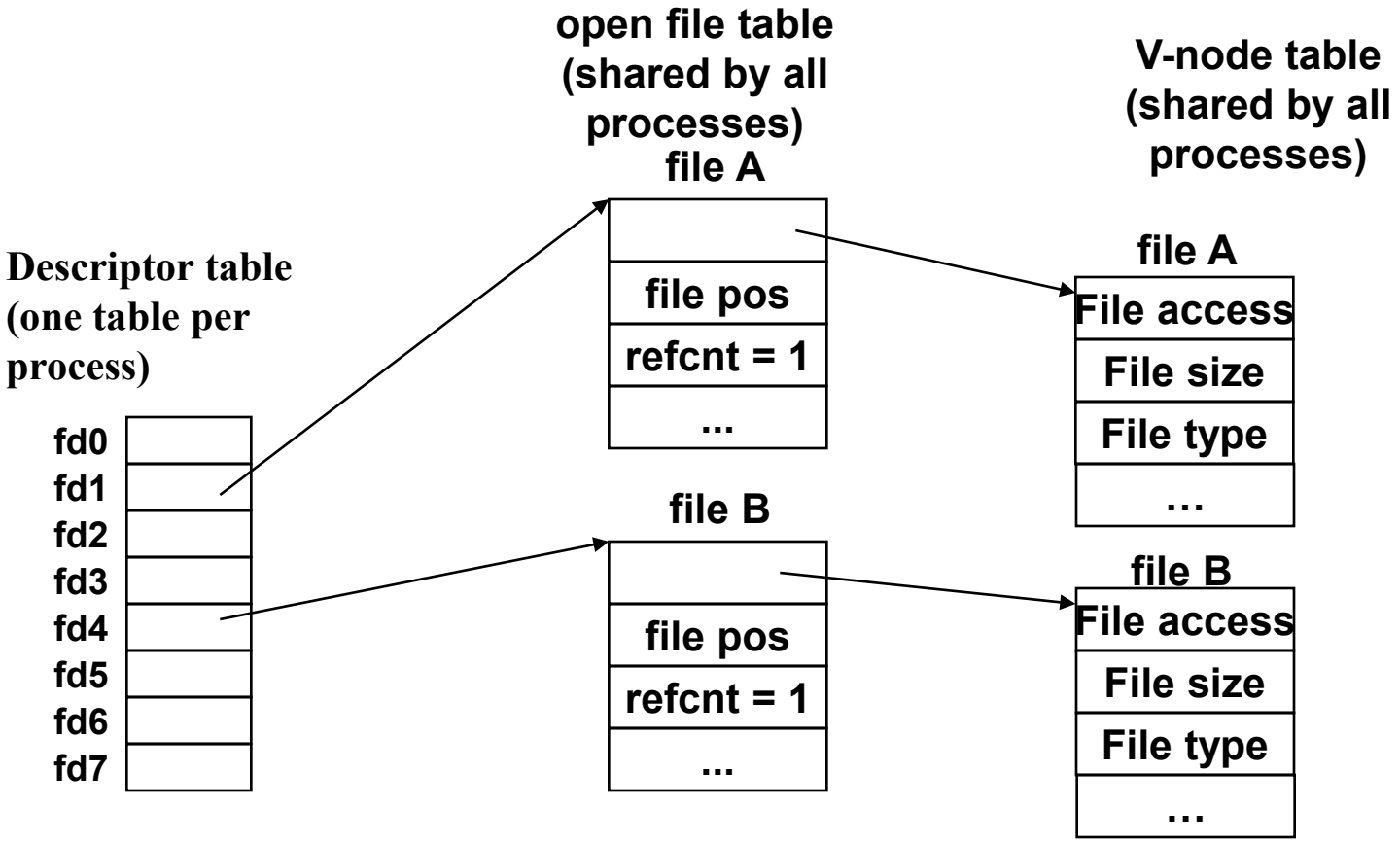
- The Linux kernel
  - organizes all files in a single *directory hierarchy*
    - anchored by the *root directory* named / (slash)
  - Each file in the system is a direct or indirect descendant of the root directory



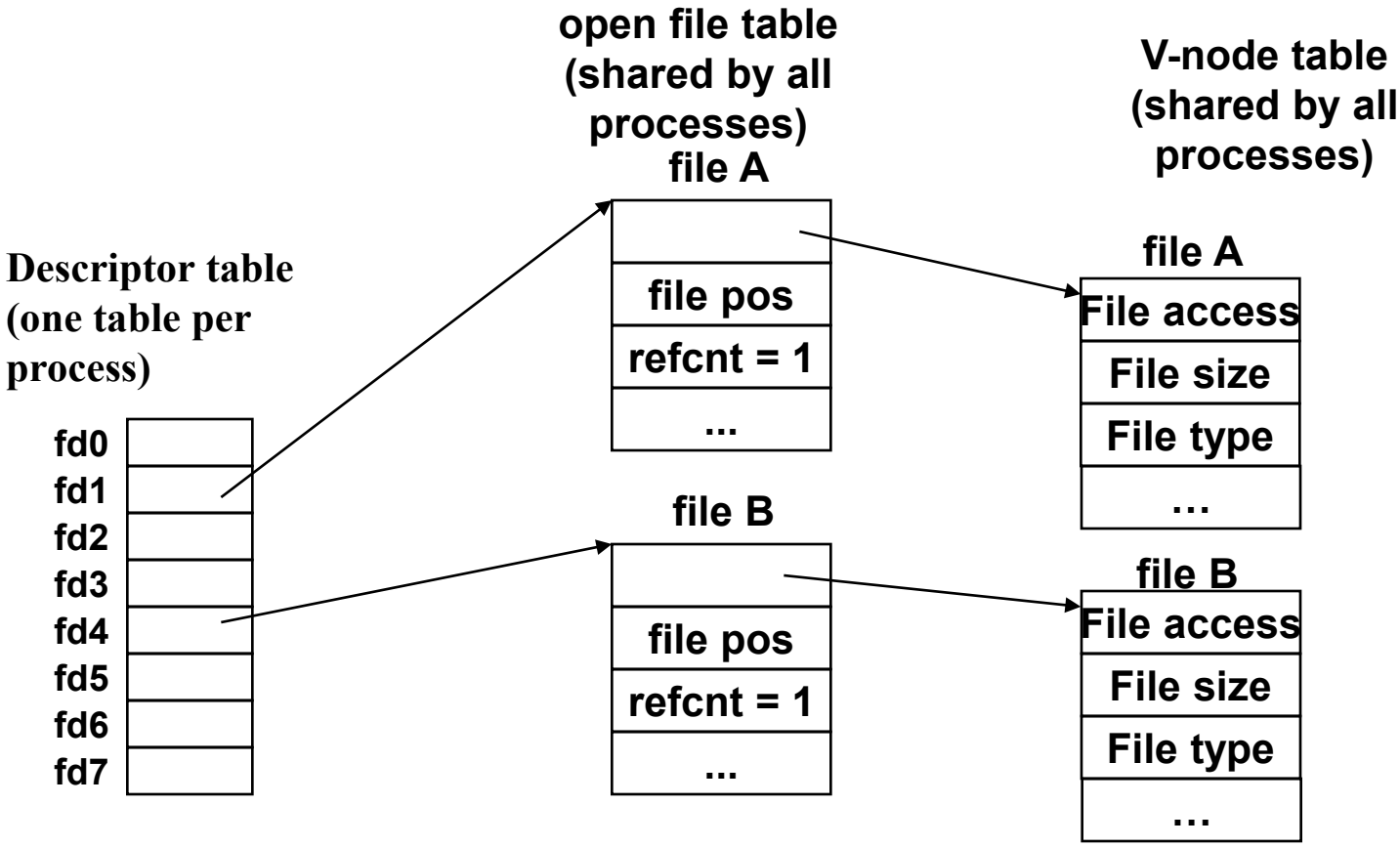
Kernel Data Structures for Files



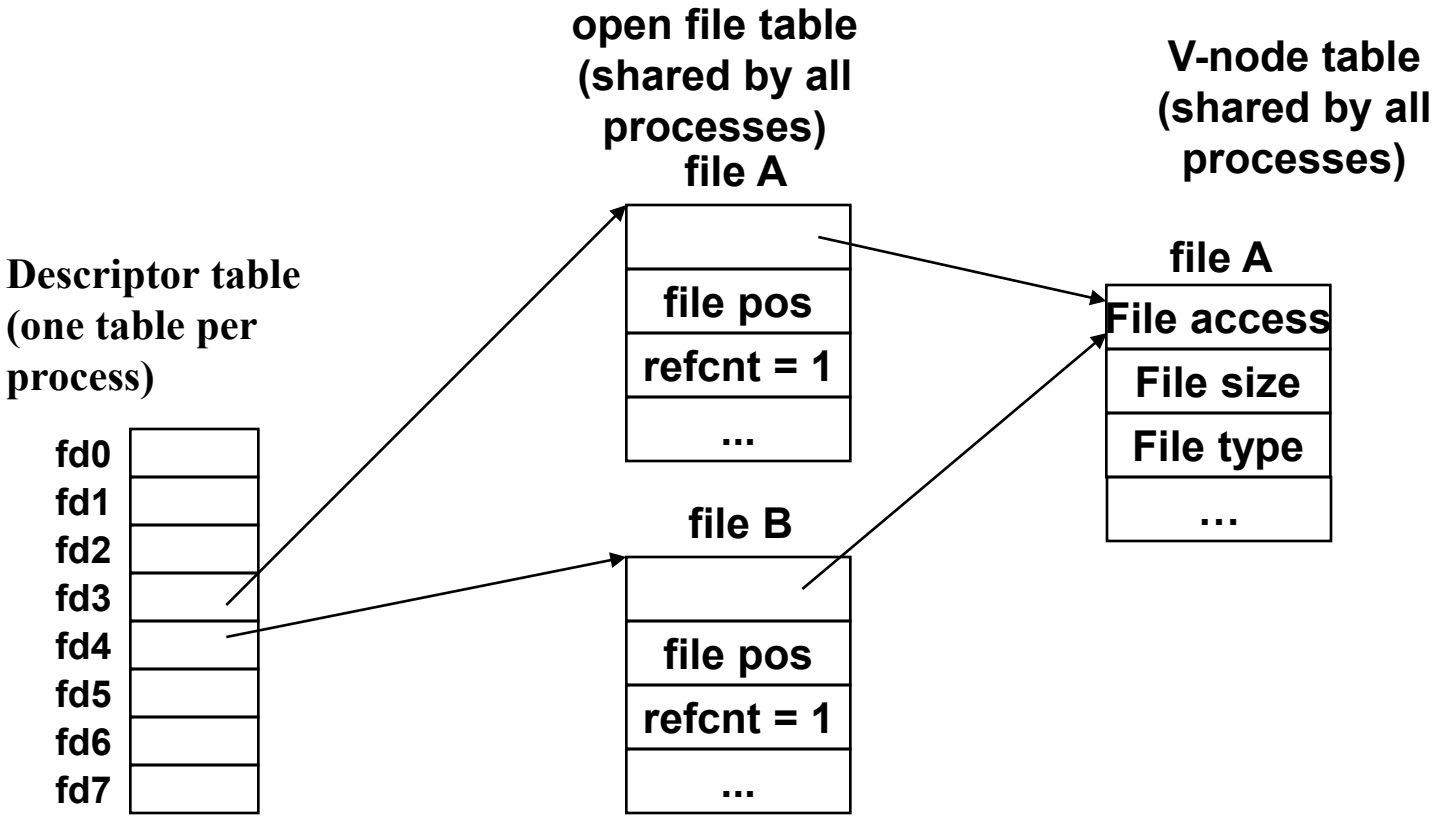
Kernel Data Structures for Files



V-Node table

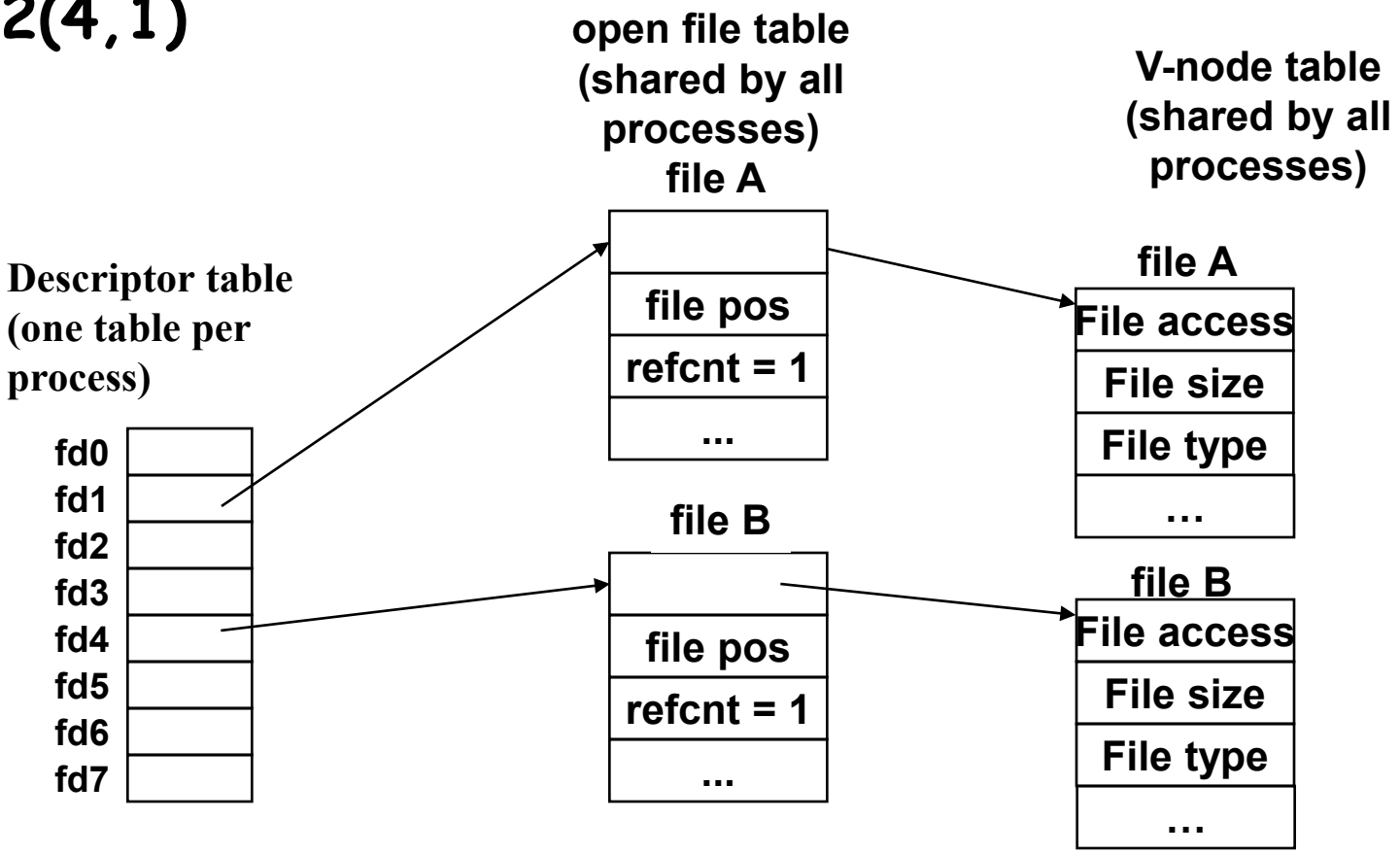

open file table
(shared by all
processes)
file A
V-node table
(shared by all
processes)
Descriptor table
(one table per
process)
fd0
fd1
fd2
fd3
fd4
fd5
fd6
fd7
file pos
refcnt = 1
...
file A
File access
File size
File type
…
file B
file pos
refcnt = 1
...
file B
File access
File size
File type
…

Sharing Files



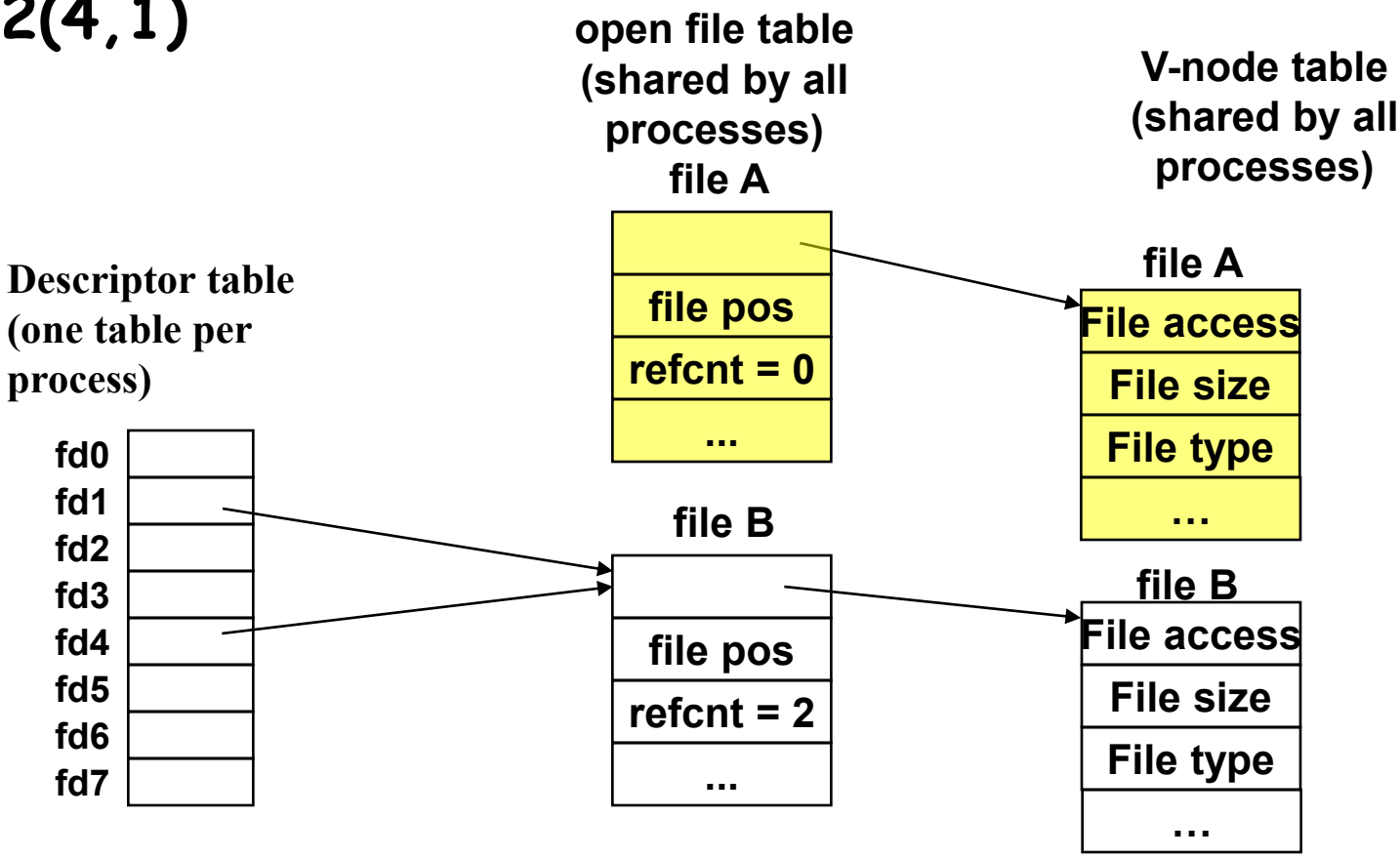
Redirection

dup2(4,1)

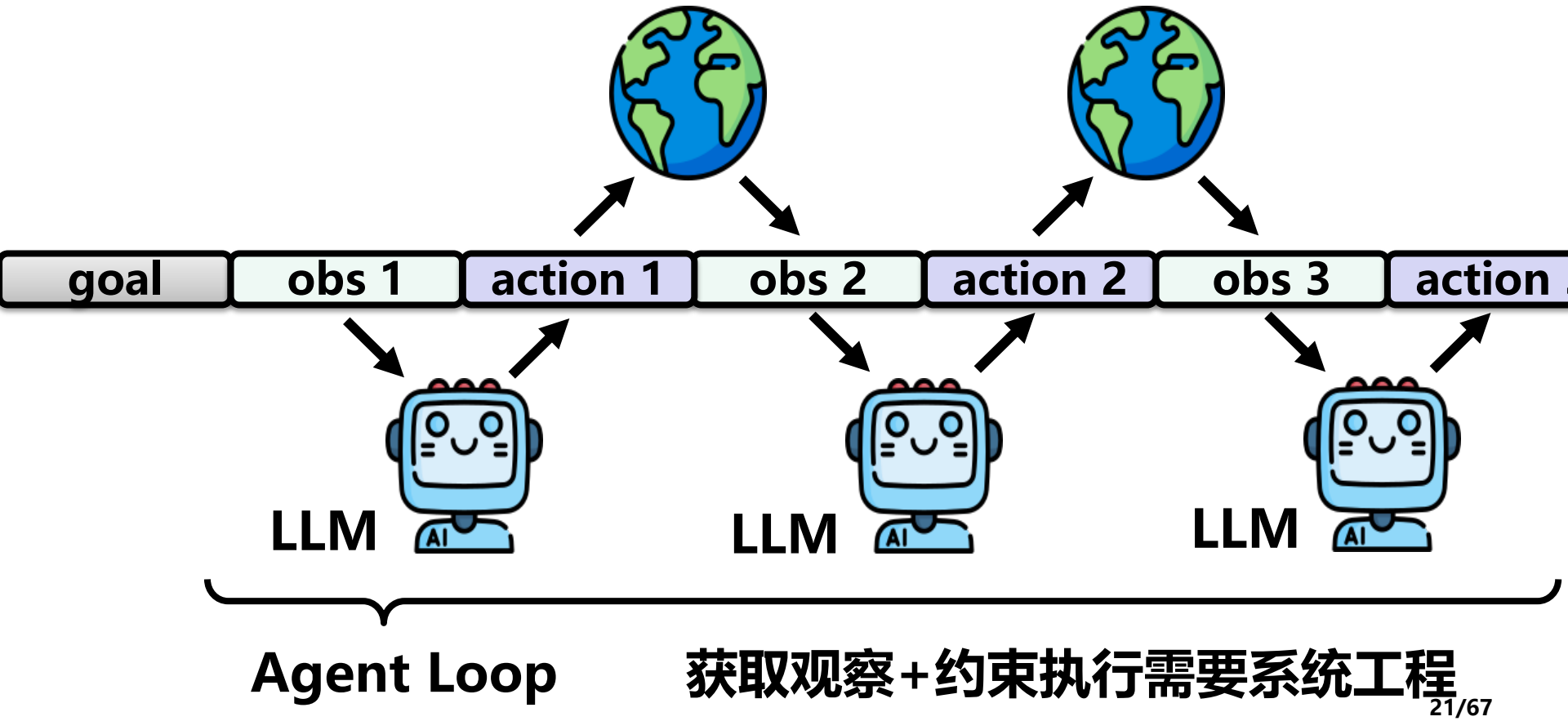


Redirection

dup2(4,1)



AI Agent = LLM + Harness



智能体记忆与CSAPP

- 层次化存储
- 文件系统
- 虚拟内存

"存不下怎么办"的三个朴素方案

- 方案 1："分层存"
  - 不是所有东西都放在一起，按"快但小""慢但大"分成几层
- 方案 2："按需取"
  - 要用的时候才加载进来，只把当前用得着的部分读进来
- 方案 3："扔旧的"
  - 旧的、不常用的先踢出去，给每个信息打"新旧"标签

- 系统思想关键词：分层（hierarchy）、按需加载（on-demand）、替换（replacement）、压缩（compression）

智能体工具与CSAPP

- 工具调用协议MCP、CLI（calling convention）
- 启动和运行工具（进程）
- 调用远程工具（网络）
- 安全性（系统权限管控）
- 鲁棒性（speculative execution)

多智能体协作优势

- 专业分工：各司其职，效果更精准
- 降低复杂度：任务拆解，上下文更清晰
- 并行高效：多任务同时推进，响应更快
- 相互校验：互审互评，减少幻觉与错误
- 灵活可扩展：按需增减角色，架构易演进

多智能体协作适用场景

- 软件研发：写码 / 审查 / 测试 / 修复 分工协作
- 深度研究：多路并行检索 + 汇总核查
- 多视角决策：辩论、投票、方案评估
- 模拟与仿真：角色扮演、社会行为模拟

智能体工具与CSAPP

- 多进程调度
- 并发与同步
- 进程间通信

Agent Loop 四步闭环

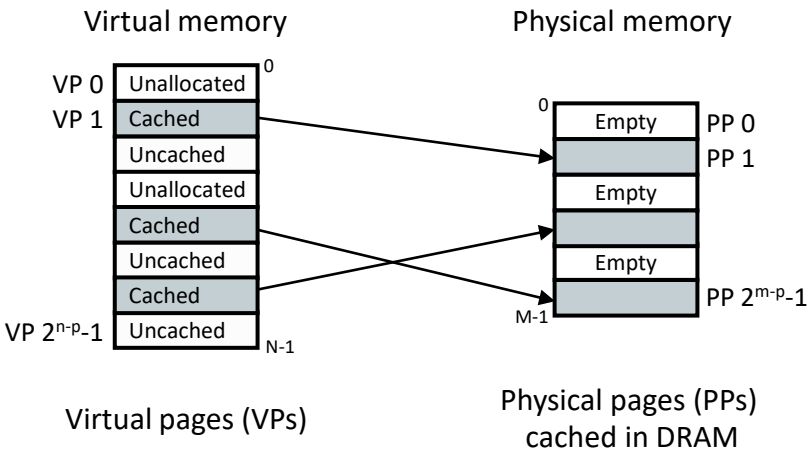
- 1. 感知（Perception）-- Agent 的感官
  - 接收用户输入 + 解析系统提示词 + 从记忆加载上下文
- 2. 认知（Cognition）-- Agent 的大脑
  - LLM 推理：分析状态、制定计划、决定下一步行动
- 3. 行动（Action）-- Agent 的手
  - 调用 API、执行代码、查询数据库（Function Calling）
- 4. 观察（Observation）-- Agent 的反馈
  - 接收工具结果 → 追加进上下文 → 判断是否完成

- 未完成 → 返回步骤 2 继续循环

Virtual Memory

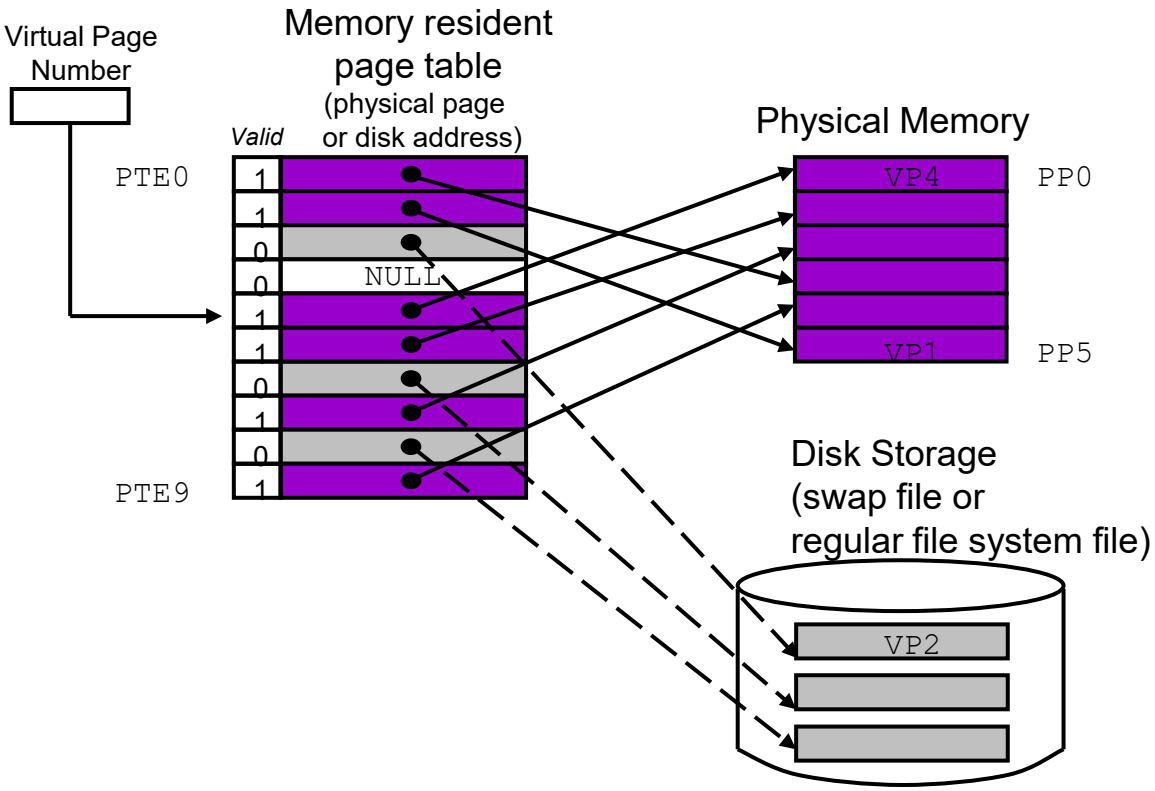
Page

- **Each page is 2^p bytes (usually, 4K) in size**
  - Serve as the transfer units between the disk and the main memory
  - virtual pages (VPs)
  - physical pages (PPs)
    - or page frames

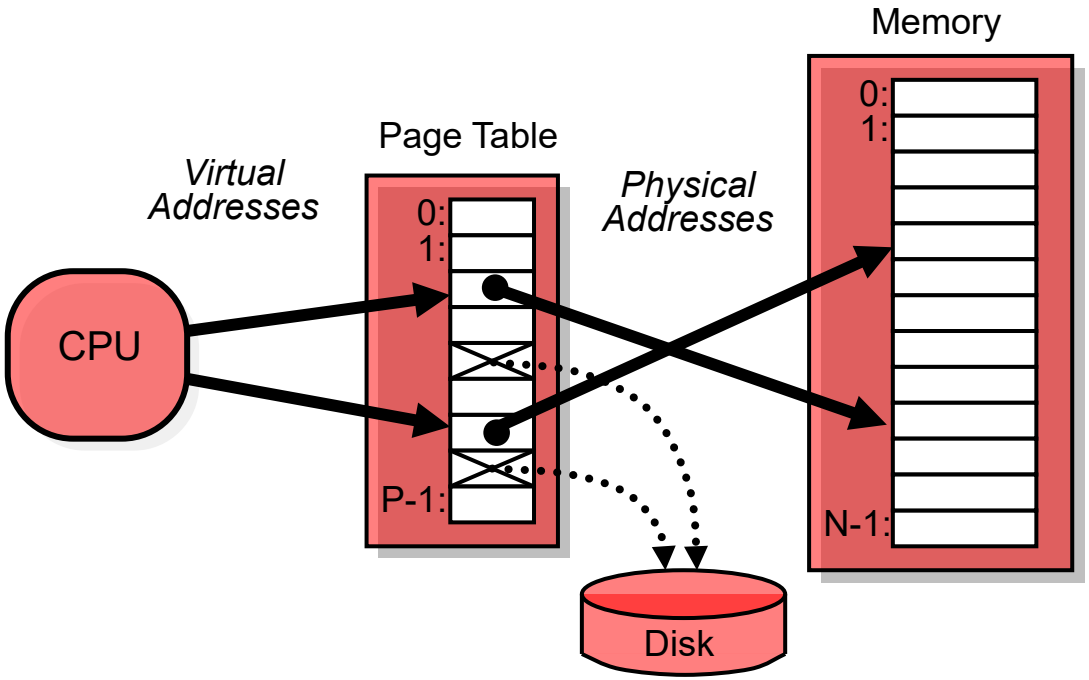


Q: Where are the uncached pages?

Page Table


Virtual Page Number
Memory resident page table
(physical page or disk address)
Valid
PTE0
PTE9
1
1
0
0
1
1
0
1
0
1
NULL
Physical Memory
VP4
VP1
PP0
PP5
Disk Storage
(swap file or regular file system file)
VP2

Page Hits



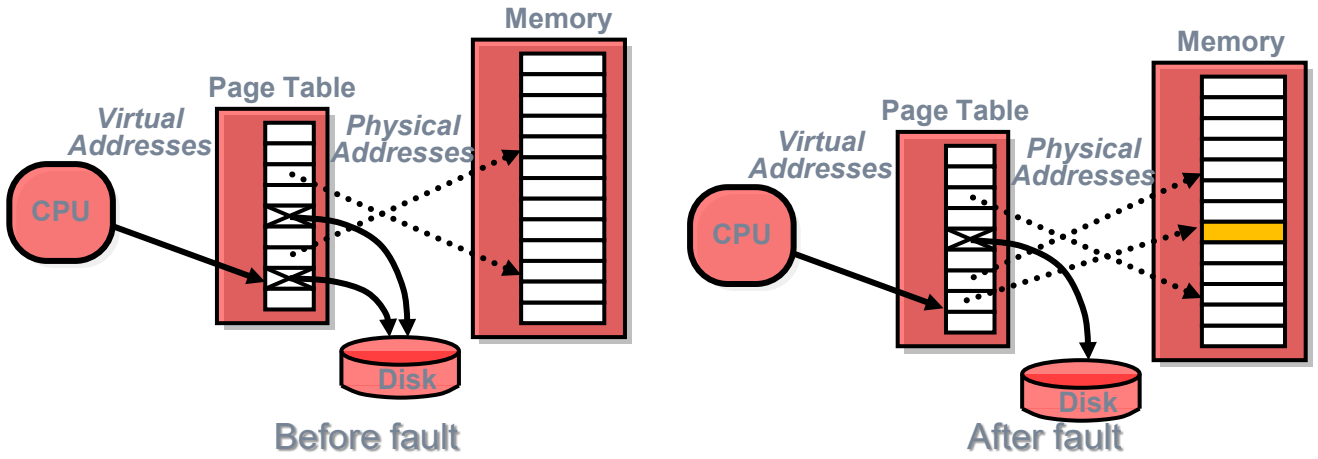
Address Translation: Hardware converts *virtual addresses* to *physical addresses* via a page table managed by OS

Page Faults

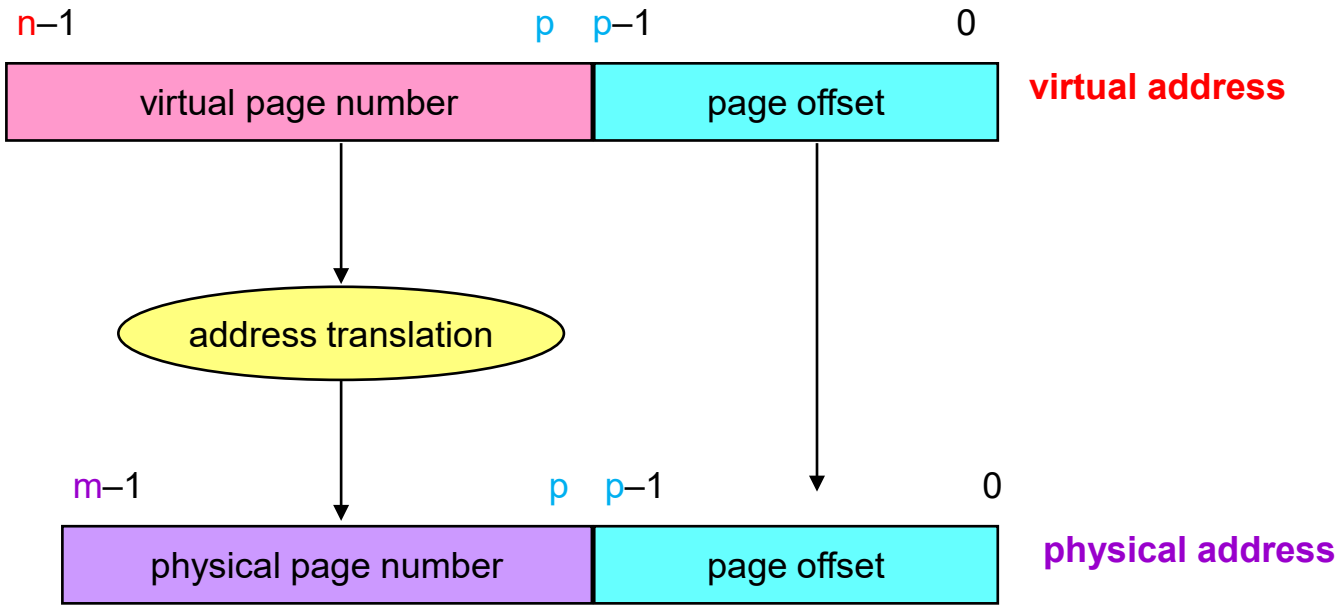
- **Page table entry indicates virtual address not in memory**
  - Three possible cases
- **Case-1: OS exception handler invoked to move data from disk into memory**
  - current process suspends, others can resume
  - OS has full control over placement, etc.

Page Faults

- **Case-1: Swapped in or paged in**
  - Swapped out or paged out
- **Case-2: Demand paging**
  - File (swapped in) or Anonymous pages (e.g., heap)
- **Case-3: Segmentation fault**

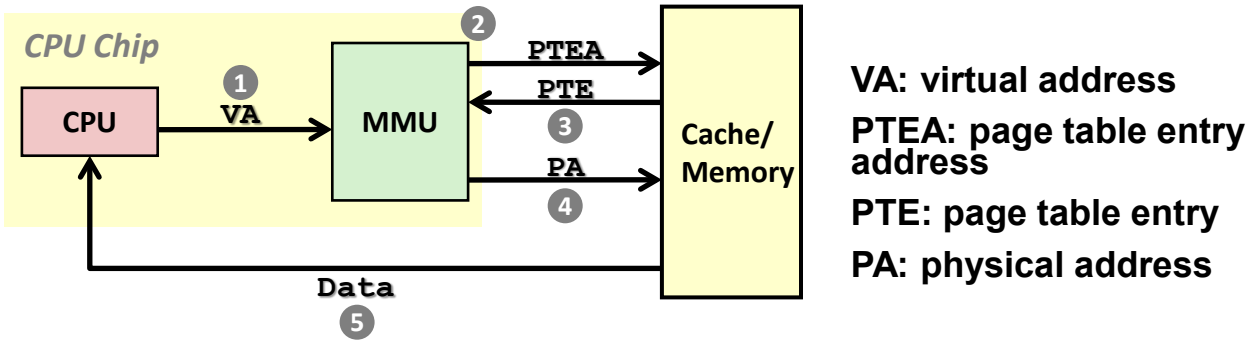


Address Translation



Notice that the page offset bits don't change as a result of translation

Page Hit



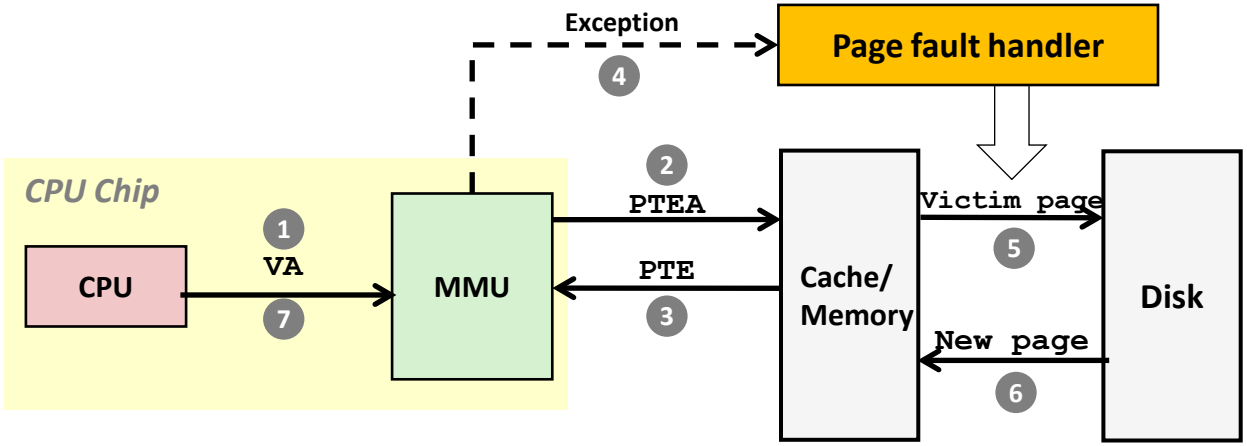
1) Processor sends virtual address to MMU

2-3) MMU fetches PTE from page table in memory

4) MMU sends physical address to cache/memory

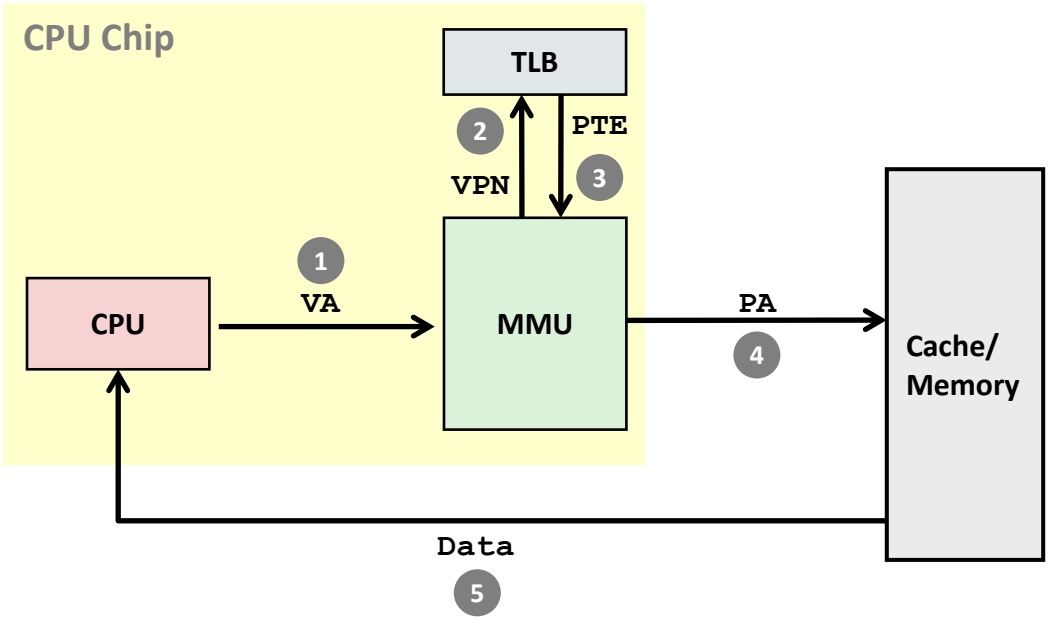
5) Cache/memory sends data word to processor

Page Faults



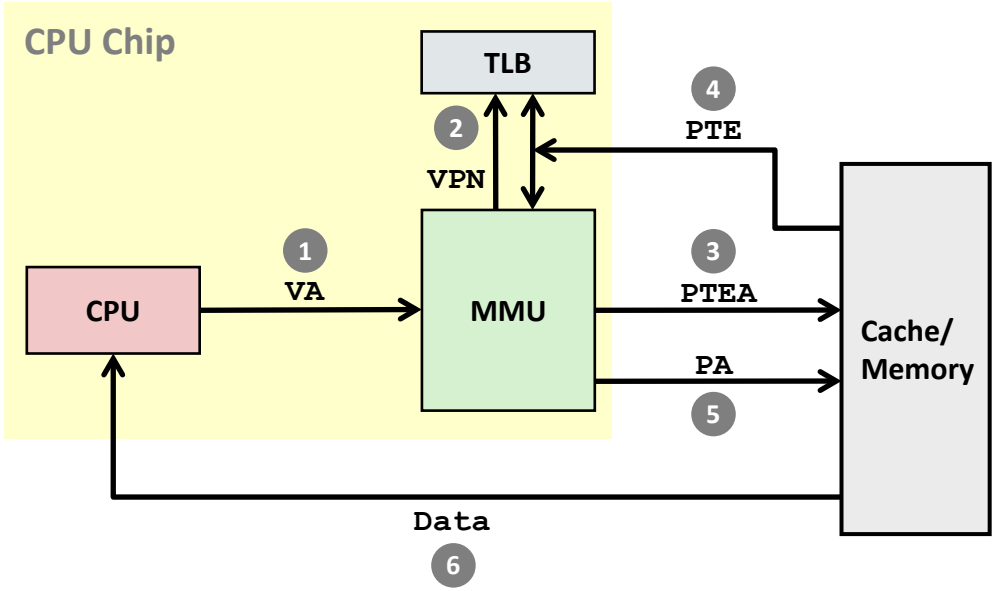
1) Processor sends virtual address to MMU
2-3) MMU fetches PTE from page table in memory
4) Valid bit is zero, so MMU triggers page fault exception
5) Handler identifies victim (and, if dirty, pages it out to disk) (optional)
6) Handler pages in new page and updates PTE in memory
7) Handler returns to original process, restarting faulting instruction

TLB Hit



A TLB hit eliminates a memory access

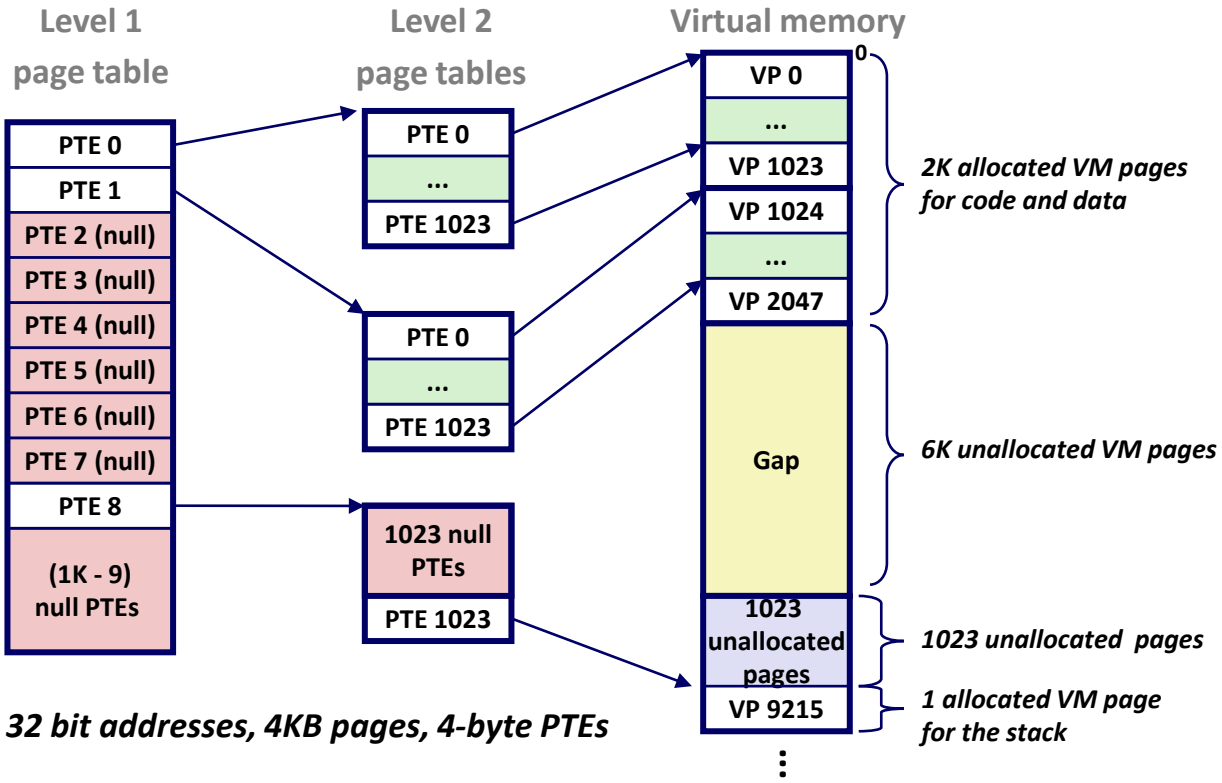
TLB Miss



A TLB miss incurs an additional memory access (PTE)

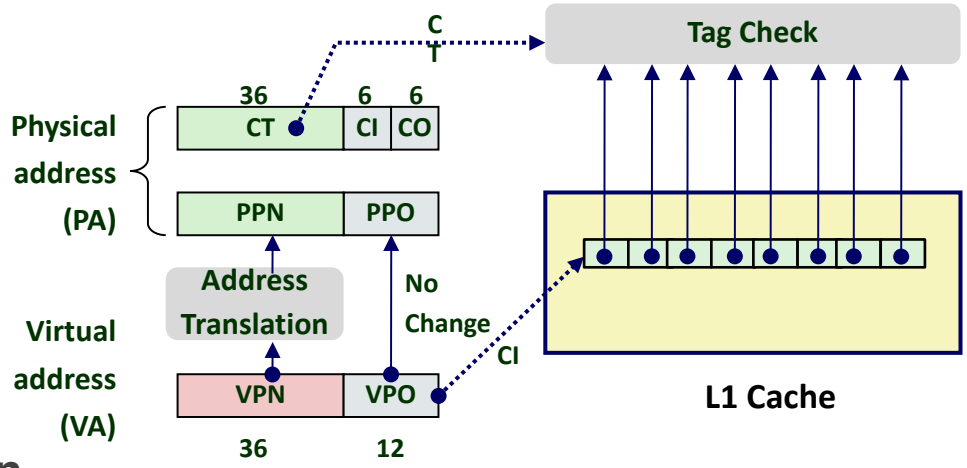
Q: Fortunately, TLB misses are **usually not frequently**. Why?

Multi-Level Page Tables



X86-32: *32 bit addresses, 4KB pages, 4-byte PTEs*

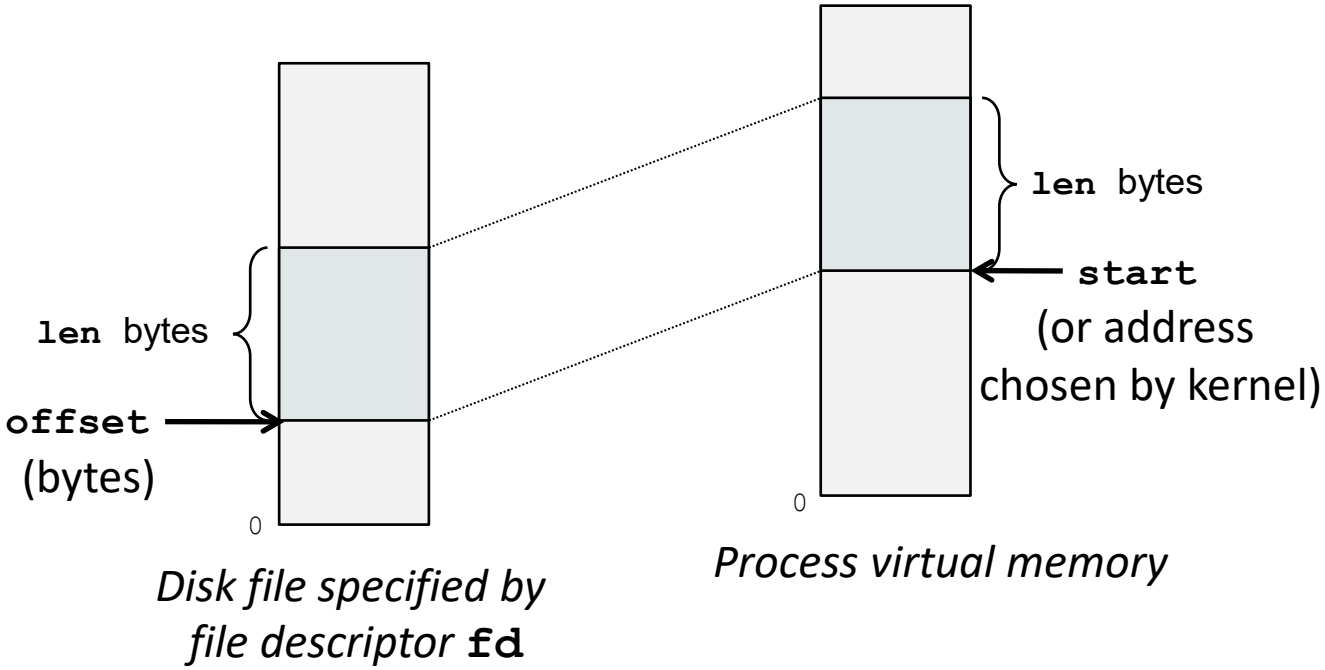
Cute Trick for Speeding Up L1 Access (VIPT)



- **Observation**
  - Bits that determine CI identical in virtual and physical address
  - Can index into cache while address translation taking place
  - Generally we hit in TLB, so PPN bits (CT bits) available next
  - “Virtually indexed, physically tagged”
  - Cache carefully sized to make this possible

User-level memory mapping

```
void *mmap(void *start, int len,
           int prot, int flags, int fd, int offset)
```



mmap() example: fast file copy

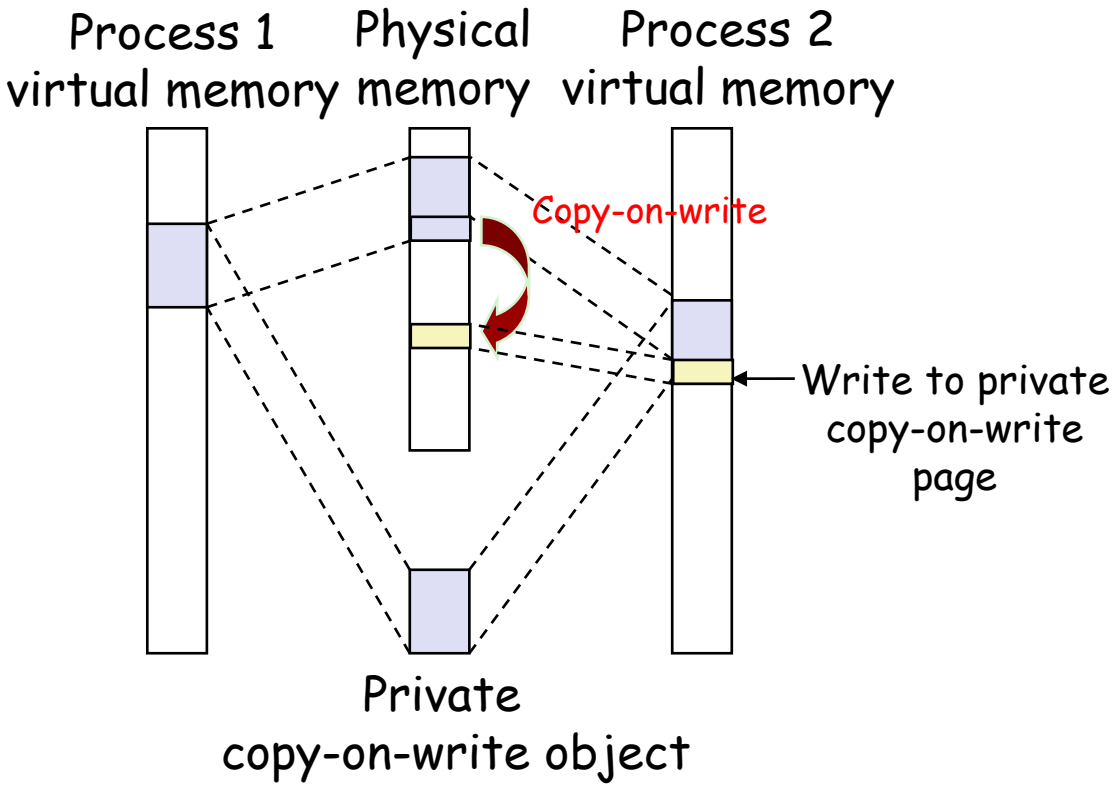
```
/*
 * mmapcopy - uses mmap to copy file fd to stdout
 */
void mmapcopy(int fd, int size)
{
  char *bufp;
  /* map the file to a new VM area */
  bufp = mmap(0, size, PROT_READ,
              MAP_PRIVATE, fd, 0);
  /* write the VM area to stdout */
  write(1, bufp, size);
  return ;
}
```

mmap() example: fast file copy

```
int main(int argc, char **argv)
{
  struct stat stat;
  /* check for required command line argument */
  if ( argc != 2 ) {
    printf("usage: %s <filename>\n", argv[0]);
    exit(0) ;
  }
  /* open the file and get its size*/
  fd = open(argv[1], O_RDONLY, 0);
  fstat(fd, &stat);
  mmapcopy(fd, stat.st_size);
}
```

Q: Why is this faster?

Sharing Revisited: Private COW Objects



LRU策略

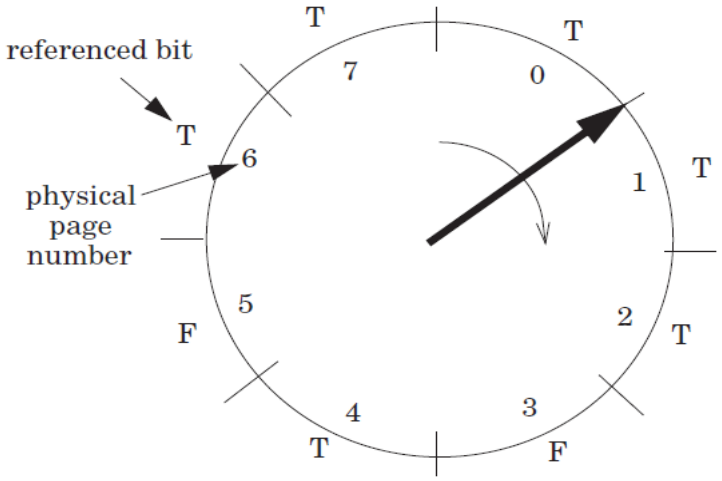
OS维护一个链表，在每次内存访问后，OS把刚刚访问的内存页调整到链表尾端；每次都选择换出位于链表头部的页面

缺点-1：对于特定的序列，效果可能非常差，如循环访问内存

缺点-2：需要排序的内存页可能非常多，导致很高的额外负载

物理页面访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
该行为链表头部越不常访问的页号离头部更近												
缺页异常（共 7 次）	是	是	否	是	是	否	是	否	是	是	否	否

时钟算法策略

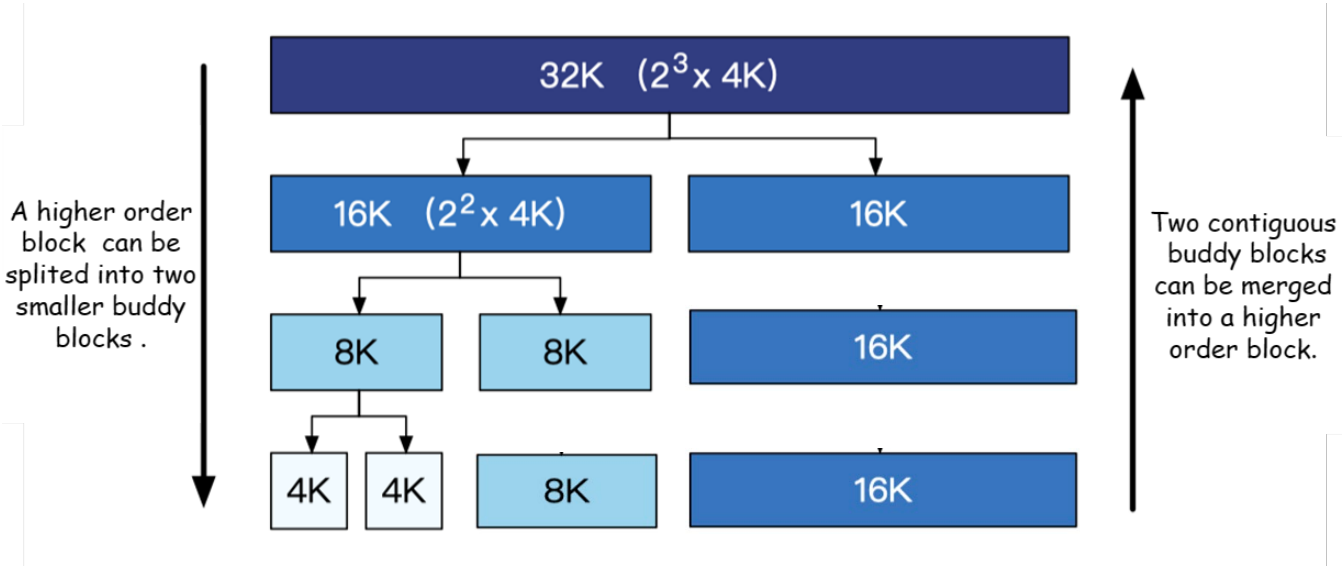


针臂：

- 指向换入位置
- 需要换出时：从下一个位置开始转

物理页面访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
	3*	3* 2*	3* 2*	3* 2* 1*	4* 2 1	4* 3* 1	4* 3* 5*	4* 3* 5*	2* 3 5	2* 3* 5	2* 3 4*	2* 3* 4*
缺页异常	是	是	否	是	是	是	是	否	是	否	是	否

Buddy Systems



Recursively split and merge

OS kernel uses this way to allocate/free physical pages

Problem 3: Virtual Memory (39 points)

The following system specifications apply to all sub-problems unless stated otherwise. These match a simplified x86-64 configuration from the lecture:

- Virtual addresses are **48 bits** wide.
- Physical addresses are **52 bits** wide.
- The page size is **4 KB** (2^12 bytes).
- The system uses a **4-level** page table with **9 index bits per level**.
- Each page table entry (PTE) is **8 bytes**.
- The L1 data cache is **32 KB**, organized as **64 sets × 8-way × 64-byte lines**.

1. Bit-width fundamentals (8 pts)

Fill in the following table.

Quantity	Value
VPO / PPO bits	[a]
VPN bits	[b]
PPN bits	[c]
Entries per page-table page	[d]
Index bits per page-table level	[e]
Cache offset (CO) bits	[f]
Cache index (CI) bits	[g]
Cache tag (CT) bits (of physical address)	[h]

2. Page-table memory with regular and huge pages (15 pts)

A process maps a single **contiguous 8 MB** region starting at virtual address `0x0000_0000_0040_0000`. No other virtual pages are mapped. Each page-table page is 4 KB and holds 512 PTEs.
The 48-bit virtual address is decomposed as:

Field	L1 (PML4)	L2 (PDPT)	L3 (PD)	L4 (PT)	VPO
Bit range	47–39	38–30	29–21	20–12	11–0
Width	9 bits	9 bits	9 bits	9 bits	12 bits
What it selects	PML4 entry	PDPT page	PD page	PT entry	Byte within page

a. Using 4 KB regular pages, how many page-table pages are needed at each level? Fill in the table and give the total page-table memory in KB. Show which indices are populated at each level. (10 pts)
Hint: first compute the L1–L4 indices of the start address `0x0000_0000_0040_0000` and the end address `0x0000_0000_00BF_FFFF`, then determine how many distinct L3 and L4 entries are needed.

Level	Populated indices	Page-table pages
L1 (PML4)	[a1]	[a2]
L2 (PDPT)	[b1]	[b2]
L3 (PD)	[c1]	[c2]
L4 (PT)	[d1]	[d2]
Total		[e]

Total page-table memory: [f] KB

b. x86-64 supports 2 MB huge pages: a Level-3 (PD) entry can set the PS (Page Size) bit and directly map a 2 MB physical frame, eliminating the need for a Level-4 (PT) table beneath it. How many page-table pages are needed at each level for the same 8 MB mapping using 2 MB huge pages? How much page-table memory (in KB) is saved compared to part (a)? (4 pts)

c. In one sentence, explain why multi-level page tables save memory compared to a single flat page table for a typical process that uses only a small fraction of the 48-bit virtual address space. (1 pt)

3. VIPT cache design constraint (8 pts)

Modern processors use a **Virtually Indexed, Physically Tagged (VIPT)** cache to overlap the cache lookup with TLB translation. The key requirement is:
> The bits used for cache indexing must come entirely from the page offset, so that the **virtual** and **physical** cache indices are identical.

a. Using the L1 data cache specifications above (32 KB, 64 sets, 8-way, 64-byte lines), is VIPT feasible with 4 KB pages? Show the computation. (3 pts)

b. The hardware team proposes doubling the L1 cache to **64 KB**. Two designs are on the table:

- **Design A:** double the number of sets to 128 (128 sets × 8-way × 64-byte lines).
- **Design B:** double the associativity to 16-way (64 sets × 16-way × 64-byte lines).

For each design, state whether VIPT still works with 4 KB pages. Show the computation for each. If a design breaks VIPT, state the minimum page size that would restore the condition. (5 pts)

4. Efficiency of mmap for file copying (8 pts)

Consider the following two approaches to copying an `N`-byte file to stdout.

Approach 1 — `read` + `write`:

```
1  char buf[N];
2  read(fd, buf, N);                    /* file -> user buffer */
3  write(STDOUT_FILENO, buf, N);        /* user buffer -> stdout */
```

Approach 2 — `mmap` + `write`:

```
1  char *p = mmap(NULL, N, PROT_READ,
2                 MAP_PRIVATE, fd, 0);
3  write(STDOUT_FILENO, p, N);        /* mapped region -> stdout */
```

a. Which approach copies faster? And please describe each approach's copy process, the number of copies involved, and the memory regions involved to explain the reason (use the terms: disk, kernel page cache, user buffer, kernel output buffer). (5 pts)

b. Suppose the program needs to read only a small header (the first 4 KB) from a very large file (e.g., 1 GB). Explain in 2–3 sentences why mmap is more efficient than read(fd, buf, 1GB) followed by inspecting the first 4 KB of buf. (3 pts)

Exercise: From SNAT to a Prethreaded Concurrent Server

Instructions

Answer all parts. You may use CSAPP-style wrapper functions such as `Open_listenfd`, `Accept`, `Pthread_create`, `P`, and `V`. Assume `P` and `V` are wrappers around `sem_wait` and `sem_post`.

Problem

A client inside a private LAN has socket address:

```
192.168.1.10:10086
```

It connects to an echo server at:

```
202.120.40.85:15213
```

The client reaches the server through an SNAT router. The router's public address is:

```
202.120.40.82
```

For this connection, the SNAT router creates the following NAT-table entry:

```
192.168.1.10:10086  <->  202.120.40.82:233
```

The server is implemented as a prethreaded concurrent server. One master thread repeatedly calls `Accept()` and inserts each connected descriptor into a bounded shared buffer. A fixed pool of worker threads repeatedly removes descriptors from the buffer, calls `echo(connfd)`, and closes the connection.

Part 1. Sockets Interface

Explain the role of the sockets interface in this client-server transaction.
In particular:

1. What are the purposes of `connect`, `bind`, `listen`, and `accept`?
2. What is the difference between a listening descriptor `listenfd` and a connected descriptor `connfd`?
3. What socket pair identifies the TCP connection after the server accepts it?

Part 2. SNAT and IP Forwarding

Using the NAT-table entry above, identify the source and destination socket addresses at each point:

1. When the packet leaves the private client, before it reaches the SNAT router.
2. After SNAT, when the packet is forwarded on the public Internet.
3. As seen by the server.
4. On the return path, before the router translates the packet back.
5. On the return path, after the router translates the packet back into the LAN.

Briefly explain why the server cannot directly see `192.168.1.10:10086`.

Part 3. Bounded Descriptor Buffer

The prethreaded server uses a bounded FIFO buffer to pass connected descriptors from the master thread to worker threads.

Complete the synchronization design:

1. Define the buffer state and the semaphores needed.
2. Give the initial value of each semaphore.
3. Write pseudocode for `sbuf_insert()` and `sbuf_remove()`.
4. Explain why the `P` operations must be ordered carefully.

Part 4. Prethreading Advantages

Explain why a prethreaded server can outperform:

1. An iterative server.
2. A thread-per-connection server.

Your answer should discuss both concurrency and resource-management overhead.

Part 5. Performance Trade-offs

The server uses `N` worker threads and a descriptor buffer of size `B`.

Discuss two trade-offs in choosing `N` and `B`. Your answer should mention at least two of the following: CPU cores, I/O blocking, context-switch overhead, memory use, bursty arrivals, queueing delay, and synchronization overhead.

Part 6. Fairness and Starvation

Does the semaphore-based bounded buffer guarantee fairness among clients or worker threads? Could starvation happen?

Explain:

1. What FIFO ordering the buffer provides.
2. What ordinary POSIX semaphores do not guarantee.
3. One way to improve fairness if the server requires it.